

Contents

ScrollSense — Assignment 1	2
Deliverable A — Assumptions and Scope	2
A.1 Assumptions	2
A.2 Descoping	4
A.3 Revision log	4
Deliverable B — Conceptual Design	5
Step 1 — Entity inventory	5
Step 2 — Relationship inventory	7
B.1 — ER diagrams	8
B.2 — Temporal register	14
B.3 — The three hardest decisions	14
Deliverable C — Logical Design and Relational Algebra	15
C.1 — Relations	15
C.2 — Mapping decision log	21
C.3 — Relational algebra	23
Deliverable D — Normalisation	24
D.1 Relation 1 — ImpressionSegment (telemetry)	24
D.1 Relation 2 — CreatorTierFull (temporal)	25
D.1 Relation 3 — JudgeScoreFull (agent layer)	26
D.1 Relation 4 — VideoTagged (content)	27
Where 3NF was kept instead of BCNF	27
D.2 — The denormalisation I did not do	28
Deliverable E — Physical Implementation	28
E.1 — Schema notes	28
E.2 — Constraint trace table	30
E.3 — Data generator	32
E.4 — JSON, used once	33
Deliverable F — Queries	34
Summary	34
F1 · Top 10 audio tracks by distinct videos in the last 7 days	34
F2 · Watch hours and completion rate per creator, live clips only	35
F3 · Videos with no audio track, with NOT IN	36
F3 · The same question, with NOT EXISTS	36
F4 · Users who liked then retracted within 60 seconds	36
F5 · Videos whose caption carries a given hashtag	37
F6 · Users shown a creator's clips who never engaged	37
F6 · A signal roll-up with UNION	38
F6 · The same roll-up with UNION ALL	38
F7 · Cost of each agent session last month, by prompt template version	38
F8 · Videos whose moderation state changed more than twice	39
F9 · Each user's longest streak of consecutive active days	39
F10 · Creators by 7-day rolling watch time, with week-over-week change	40
F11 · Full nesting tree for one session's tool calls, with depth	41
F12 · Sessions where a recommended clip was watched to completion	42
F13 · Turns the judge scored above 4 but the user rated down	42
Deliverable G — Views and Transactions	43
G.1 — View specification	43
G.2 (a) — Updatability	44
G.2 (b) — The as-of problem	44

G.3 — Transactions	45
Appendix — Use of an LLM assistant	46
Where I used one	46
Where it was wrong	46
Two things the assistant was right about that the task sheet was not	47

ScrollSense — Assignment 1

Rohan Sai P. · DA24B049 · Data Management · Due 9 September 2026

Generator seed: 49 (from roll number DA24B049). SQLite version used: 3.53.4 (checked with `SELECT sqlite_version();`).

Deliverable A — Assumptions and Scope

A.1 Assumptions

#		The ambiguity, as a Where question	My assumption	Why	What breaks if the opposite were true
A1	2.4	Is an impression with no view a row, or no row?	A row. Every clip entering the viewport is recorded whether or not it becomes a view.	The brief says most impressions never become views, and you can't measure a leak you didn't record.	We would only have rows for clips people watched. We could count how many finished a clip but not how many were shown it, so per-creator completion reports become impossible.
A2	2.4	One impression, how many view segments?	Zero or more. A segment is one continuous stretch of playback, with wall-clock bounds and playhead offsets. Segments never overlap in wall-clock time but can repeat in playhead position. Watch time lives only here.	The user scrolls away and returns, or the clip loops, so watch time can exceed clip duration.	A single <code>watch_ms</code> column on Impression sums the loops together and you can no longer tell watch time from completion.
A3	2.4	What counts as reaching the end?	A segment reaching 95% of duration. An impression is complete if any segment did.	The brief separates completion from watch time but never defines it, and players rarely report an exact 100%.	Total watch time $\geq$ duration means watching 60% twice counts as a completion.
A4	2.4	Is a like state you update, or an event?	An event. Likes and retractions are append-only, one row per act. Current state is the latest row.	The brief says a retraction is a signal in its own right, and 2.7 warns against writing the same fact twice.	Keeping a like row you update plus a separate retraction row means two writes for one action. If the process dies between them, you get a like marked as ended with no retraction recorded, or the reverse.
A5	2.4	One share to two destinations, one row or two?	Two.	Destination belongs to the act of sharing, not to the clip.	Destination is multivalued in one row, which breaks 1NF.

#		The ambiguity, as a Where question	My assumption	Why	What breaks if the opposite were true
A6	2.5	Must a turn have both messages?	One user message, at most one assistant message.	Tool calls can error, so a turn can end without a reply.	Failed turns can't be stored at all.
A7	2.5	Can a tool call belong to more than one turn?	No. One turn per call, nesting via a nullable <code>parent_call_id</code> .	A call is made while producing one assistant message, and the agent re-executes rather than reusing.	You need a junction table and the nesting is no longer a tree.
A8	2.5	Is a recommended clip the same as an impression?	No. A recommendation is clip X at position N of turn T. It becomes an impression only if the user scrolls it into view.	The brief wants recommendations traced to what happened next, which needs the two kept apart.	Every recommendation counts as seen, so the question answers itself.
A9	2.5	Can a user rate a turn more than once?	Yes, append-only with timestamps, latest wins. Most turns have none.	Very few turns are rated, and a user changing their mind is itself information.	A thumbs flip overwrites the original and looks identical to a turn disliked from the start.
A10	2.5	Can a turn have more than one judge score?	Yes, one per judge version, keyed <code>(turn_id, judge_version_id)</code> .	Judges get upgraded, and comparing old against new needs turns scored by both.	Upgrading the judge overwrites history, so a change in the scale looks like a change in quality.
A11	2.5	Is a prompt template one row you edit?	No. Many immutable versions, and a turn points at a version.	The brief says responses must stay linked to the exact text used at the time.	Editing a template rewrites what every past turn was answered with.
A12	2.3	Does a block delete a follow or end it?	Ends it, with a timestamp and a reason.	Section 3 requires knowing who followed whom at a past moment.	Deleting means March's data can't show a follow that existed in March.

Appendix to A.1 — further assumptions

#		The ambiguity, as a Where question	My assumption	Why	What breaks if the opposite were true
A13	2.1	“Changeable at most twice a year” — calendar year, anniversary year, or rolling 365 days?	Rolling 365 days, counted at the instant of the attempted change.	A reset date lets someone change twice in December and twice in January.	Calendar year is easier to count but reopens that loophole. Either way handle changes need timestamps, and neither version fits in a CHECK.
A14	2.2	What happens to a track when its original video is taken down?	The track survives; <code>origin_video_id</code> goes NULL.	One takedown shouldn't break 50,000 unrelated clips.	Cascading deletes other creators' content. Note the video and track reference each other, so creation order matters.
A15	2.4	What happens to a deleted user's comments on other people's videos?	Retained, with the account hidden rather than the comment removed.	The video owner's engagement history shouldn't change because a commenter left.	Cascade removes engagement from clips whose owners did nothing wrong; anonymising loses the link back to the signal stream.

#		The ambiguity, as a Where question	My assumption	Why	What breaks if the opposite were true
A16	2.2	Can one video carry more than one audio track? (given in the task sheet as a worked example)	Exactly zero or one.	Feed rendering plays a single track and the UI has no mixer; 2.2 also says “an optional audio track”, singular.	Video–AudioTrack becomes M:N and needs a junction table, probably with a layer or ordering attribute.

A.2 Descoping

D1 — Comment content and threading

Not building: the comment body, replies to comments, likes on comments. A comment is recorded as a signal event only: this user commented on this clip at this time.

What the brief seemed to ask for: 2.4 lists comment among the explicit signals, and the task sheet’s gap list asks what happens to a deleted user’s comments on other people’s videos, which suggests comments are stored objects.

Why the cost exceeds the value in Module 1: comment text brings its own moderation states, its own reply nesting, and its own reporting flow. That is a second content system beside the video one, and none of the five consumers in 2.6 asks for it. The signal is what the ML team needs; the text is not.

Re-entry path: add a Comment table with a nullable `parent_comment_id` for replies, and point the existing comment signal row at it. No change to any current table.

D2 — The ranking model registry

Not building: anything about the models themselves. Impressions record which model version ranked the clip into position, and that stays a plain identifier with no table behind it describing training runs, features, or deployment windows.

What the brief seemed to ask for: 2.4 says every impression carries the model version that ranked it there, and the opening complaint is that nobody can tell whether quality dropped after v14.

Why the cost exceeds the value in Module 1: answering the v14 question needs the version stamped on the impression, which we have. Modelling what a version is made of is an ML platform concern, not a data model one, and the brief says we are not building the ML pipeline.

Re-entry path: create a ModelVersion table and turn the existing identifier column into a foreign key. Every impression already carries the value.

D3 — The audio licensing catalogue

Not building: rights holders, territories, royalty rates, licence expiry. A track is marked as original or licensed.

What the brief seemed to ask for: 2.2 says a track is either original to some video or licensed from a catalogue, which suggests a catalogue exists somewhere.

Why the cost exceeds the value in Module 1: the functional requirement in the brief is that tapping a sound lists every clip using it, and that works from the track alone. Licensing is a finance and legal system with its own lifecycle, and no consumer in 2.6 queries it.

Re-entry path: add a Licence table keyed by track, holding rights holder and validity dates. The origin flag on AudioTrack stays as it is.

A.3 Revision log

Stage 1 and Stage 2 were optional and I did not take up the written feedback. This log records my own design revisions during the build rather than responses to feedback.

#	Where	What changed	What forced the change
R1	A.1	A4 moved from a mutable like row with an <code>ended_at</code> , plus a separate retraction row, to a single append-only event stream.	Two writes for one action can half-complete, which is the failure described in 2.7. A single log removes the second write, and it also keeps repeated like/retract cycles, which the mutable version loses after the first.
R2	B.1	Block and Mute changed from event entities to temporal ones, with <code>started_at</code> and <code>ended_at</code> .	A block can be lifted, so the fact holds over an interval rather than at an instant. Leaving them as events would have needed a separate unblock event and a scan to pair them up. Follow was already temporal, so this also made the three social relations consistent.
R3	C.1	Recommendation gained a surrogate <code>recommendation_id</code> , with <code>(turn_id, position)</code> kept as a <code>UNIQUE</code> constraint.	Impression has to point back at the recommendation it came from. With the natural composite that would have meant two nullable foreign key columns on the largest table in the design.
R4	E.1	Moderation state, creator tier and share destination moved from <code>CHECK</code> constraints to lookup tables, adding three tables not present in C.1.	Each set is owned outside engineering and each value carries attributes a <code>CHECK</code> cannot hold, such as whether a moderation state still shows in the feed.
R5	E.1	Handle uniqueness was made stricter than the brief asks: unique across all accounts rather than only active ones.	A <code>UNIQUE</code> index cannot be limited to a subset of rows. Reserving the handle of a deactivated account also avoids an account restored inside its 30-day window finding its handle taken.
R6	E.1	Impression, Signal, Follow and AgentSession changed from <code>ON DELETE CASCADE</code> to <code>ON DELETE RESTRICT</code> on their foreign key to AppUser.	Cascading would destroy watch history, signals, the social graph and session costs on a single user delete, which is what section 3 and Finance both say must survive. The database now refuses the hard delete, leaving the soft path the brief describes as the only route.
R7	E.3	The generator stopped using <code>INSERT OR IGNORE</code> .	It was there to absorb duplicate keys, but it was also swallowing <code>CHECK</code> violations. Around 400 view segments a run were failing <code>ended_at > started_at</code> and being dropped without any error.
R8	E.3	View segment wall-clock end times are rounded up to a minimum of one second, while playhead offsets stay in milliseconds.	Timestamps are stored to second precision, so any segment shorter than a second produced <code>ended_at = started_at</code> and failed its <code>CHECK</code> . This surfaced only once R7 removed the <code>OR IGNORE</code> .
R9	F	All date arithmetic uses <code>strftime('%Y-%m-%dT%H:%M:%SZ', ...)</code> rather than <code>datetime(...)</code> .	<code>datetime()</code> returns <code>'2026-08-24 22:46:09'</code> with a space where the stored format has a <code>T</code> . Compared as text, <code>'T'</code> sorts above a space, so the obvious version of F1 pulled in 2,115 videos where the correct one returns 1,872.
R10	G.3	T1 demonstrates a block breaking follows rather than a like retraction, and the injected failure is a <code>CHECK</code> violation rather than <code>SELECT 1/0</code> .	R1 made a retraction one insert, so there is no second write to fail after. Separately, SQLite returns <code>NULL</code> for division by zero instead of raising, so <code>SELECT 1/0</code> would have let the transaction commit and demonstrated nothing.

Deliverable B — Conceptual Design

Step 1 — Entity inventory

Kinds used: **strong · weak · event · temporal · associative**. Some entities carry two kinds (for example `weak / event`) where they depend on a parent for identification and are also occurrences in time.

Users and social

Entity	One row = ...	Identifier	Kind
AppUser	one registered account	<code>user_id</code>	strong

Entity	One row = ...	Identifier	Kind
HandleChange	one handle taking effect for one user at one instant	(user_id, effective_at)	weak / event
InterestCategory	one interest category in the taxonomy	category_id	strong
DeclaredInterest	one category one user chose at signup	(user_id, category_id)	associative
InferredInterest	one category one user was assigned by the ML refresh, over one continuous interval	(user_id, category_id, valid_from)	associative / temporal
InterestSuppression	one category one user suppressed	(user_id, category_id)	associative
Follow	one user following another over one continuous interval	(follower_id, followee_id, started_at)	temporal
Block	one user blocking another over one continuous interval	(blocker_id, blocked_id, started_at)	temporal
Mute	one user muting another over one continuous interval	(muter_id, muted_id, started_at)	temporal

Creators and content

Entity	One row = ...	Identifier	Kind
Creator	one user who has become a creator	user_id	strong
CreatorTierPeriod	one creator's monetisation tier over one continuous interval	(user_id, valid_from)	weak / temporal
Video	one uploaded clip	video_id	strong
Hashtag	one distinct tag in normalised form	hashtag_id	strong
VideoHashtag	one tag appearing on one video	(video_id, hashtag_id)	associative
AudioTrack	one distinct sound usable across clips	track_id	strong
ModerationDecision	one moderation state assigned to one video at one instant	decision_id	event
Reviewer	one human reviewer or automated classifier that can decide	reviewer_id	strong

Telemetry

Entity	One row = ...	Identifier	Kind
Impression	one clip entering one user's viewport at one instant	impression_id	event
ViewSegment	one continuous stretch of playback within one impression	(impression_id, started_at)	weak / event
Signal	one explicit action one user took on one clip at one instant	signal_id	event
ShareDetail	the destination of one share signal	signal_id	weak

Agent layer

Entity	One row = ...	Identifier	Kind
AgentSession	one conversation between one user and the agent	session_id	strong
Turn	one exchange within one session	(session_id, seq)	weak
ToolCall	one invocation of one tool within one turn	(session_id, seq, call_seq)	weak / event
Recommendation	one clip returned at one position of one turn	(session_id, seq, position)	weak / associative
PromptTemplate	one named template	template_id	strong
PromptTemplateVersion	one immutable text of one template	(template_id, version_no)	weak / temporal
Model	one named model	model_id	strong

Entity	One row = ...	Identifier	Kind
ModelPrice	one model's rates over one continuous interval	(model_id, valid_from)	weak / temporal
JudgeVersion	one judge model paired with one scoring-prompt version	judge_version_id	strong
JudgeScore	one judge version's scoring of one turn	(session_id, seq, judge_version_id)	weak / associative
UserRating	one thumbs act by one user on one turn at one instant	(session_id, seq, rated_at)	weak / event

Note on identification chains. Turn depends on AgentSession for identification, and ToolCall, Recommendation, JudgeScore and UserRating each depend on Turn. Their identifiers therefore grow down the chain. Deliverable C breaks these chains with surrogate keys, turning the natural composite into a UNIQUE constraint; that mapping is recorded in the C.2 decision log.

Step 2 — Relationship inventory

Attributes are listed on a relationship only where it is M:N and the attribute describes the pairing rather than either side. Where a 1:N relationship points at an entity listed above, its attributes belong to that entity and are not repeated here.

Users and social

Relationship	From → To	Cardinality	Participation	Attributes
changed_handle	AppUser → HandleChange	1 : 1..N	both total	—
declares	AppUser → InterestCategory	0..N : 0..N	both partial	declared_at
inferred_for	AppUser → InterestCategory	0..N : 0..N	both partial	confidence, valid_from, valid_to
suppresses	AppUser → InterestCategory	0..N : 0..N	both partial	suppressed_at
follows	AppUser → AppUser	0..N : 0..N	both partial	started_at, ended_at, end_reason
blocks	AppUser → AppUser	0..N : 0..N	both partial	started_at, ended_at
mutes	AppUser → AppUser	0..N : 0..N	both partial	started_at, ended_at

Creators and content

Relationship	From → To	Cardinality	Participation	Attributes
is_creator	AppUser → Creator	1 : 0..1	user partial, creator total	—
held_tier	Creator → CreatorTierPeriod	1 : 1..N	both total	—
owns	Creator → Video	1 : 0..N	video total, creator partial	—
uses	Video → AudioTrack	0..1 : 0..N	both partial	—
originates	Video → AudioTrack	0..1 : 0..1	both partial	—
tagged_with	Video → Hashtag	0..N : 0..N	both partial	—
decided_on	Video → ModerationDecision	1 : 1..N	both total	—
decided_by	Reviewer → ModerationDecision	1 : 0..N	decision total, reviewer partial	—

Telemetry

Relationship	From → To	Cardinality	Participation	Attributes
shown_to	Video → Impression	1 : 0..N	impression total, video partial	—
saw	AppUser → Impression	1 : 0..N	impression total, user partial	—
yielded	Impression → ViewSegment	1 : 0..N	segment total, impression partial	—
came_from	Recommendation → Impression	0..1 : 0..1	both partial	—
signalled_on	AppUser → Signal	1 : 0..N	signal total, user partial	—
signal_target	Video → Signal	1 : 0..N	signal total, video partial	—
share_to	Signal → ShareDetail	1 : 0..1	detail total, signal partial	—

Agent layer

Relationship	From → To	Cardinality	Participation	Attributes
had_session	AppUser → AgentSession	1 : 0..N	session total, user partial	—
contains	AgentSession → Turn	1 : 1..N	both total	—
issued	Turn → ToolCall	1 : 0..N	call total, turn partial	—
nests_under	ToolCall → ToolCall	0..1 : 0..N	both partial	—
answered_with	Turn → PromptTemplateVersion	0..1 : 0..N	both partial	—
versioned_as	PromptTemplate → PromptTemplateVersion	1 : 1..N	both total	—
generated_by	Turn → Model	0..1 : 0..N	both partial	—
priced_as	Model → ModelPrice	1 : 1..N	both total	—
recommended	Turn → Video	0..N : 0..N	both partial	position
judged	Turn → JudgeVersion	0..N : 0..N	both partial	helpfulness, groundedness, safety, scored_at
rated	Turn → UserRating	1 : 0..N	rating total, turn partial	—

B.1 — ER diagrams

The model is drawn as four layer diagrams plus an overview. Each layer diagram stands on its own: entities belonging to another layer appear as **stubs** showing only their primary key, so every cross-layer relationship is drawn with its cardinality on at least one diagram. The overview shows which layers touch which.

Legend

Line style (crow's foot, Mermaid `erDiagram`):

Notation	Meaning
solid line	identifying relationship: the child is a weak entity and cannot be identified without its parent

Notation	Meaning
dashed line	non-identifying relationship: the child has its own key and only references the parent
\ \	exactly one (total participation)
o \	zero or one (partial participation)
\ {	one or more (total participation)
o {	zero or more (partial participation)

Entity kinds (from the Step 1 inventory; Mermaid cannot style individual boxes, so kinds are read from the inventory table and from line style):

Kind	How to recognise it
strong	own surrogate key; all relationship lines dashed
weak	identified through a parent; incoming line from its parent is solid
event	one row per occurrence; carries a single timestamp
temporal	one row per continuous interval; carries valid_from/valid_to or started_at/ended_at
associative	resolves an M:N; two solid lines, one from each parent

1. Social layer

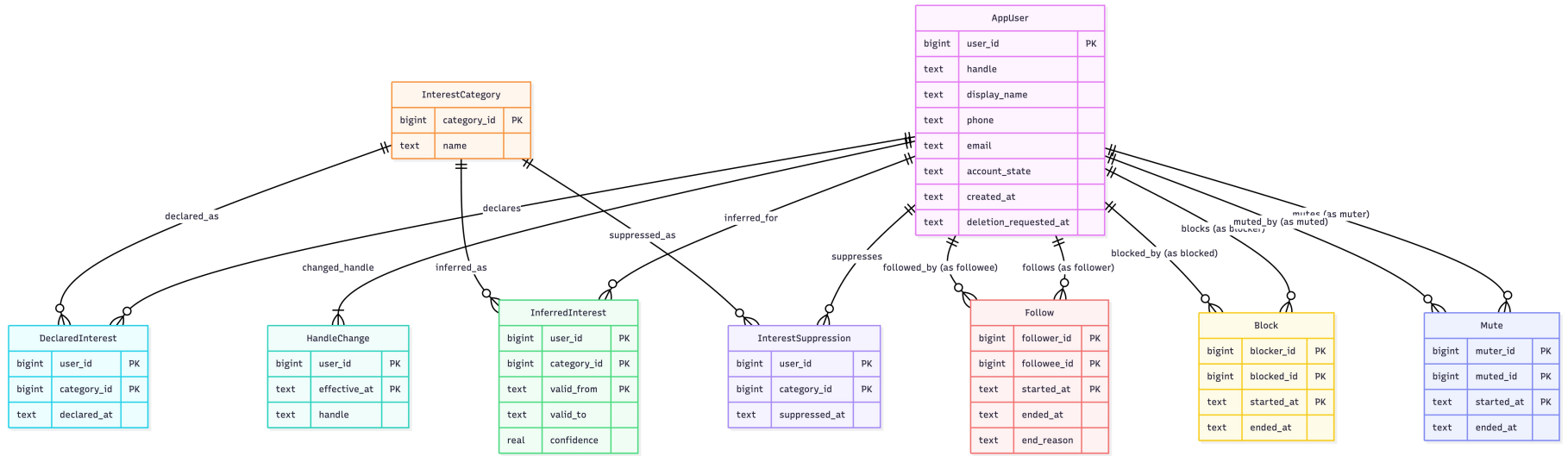


Figure 1: Social layer

No stubs: all entities on this diagram are defined here.

2. Content layer

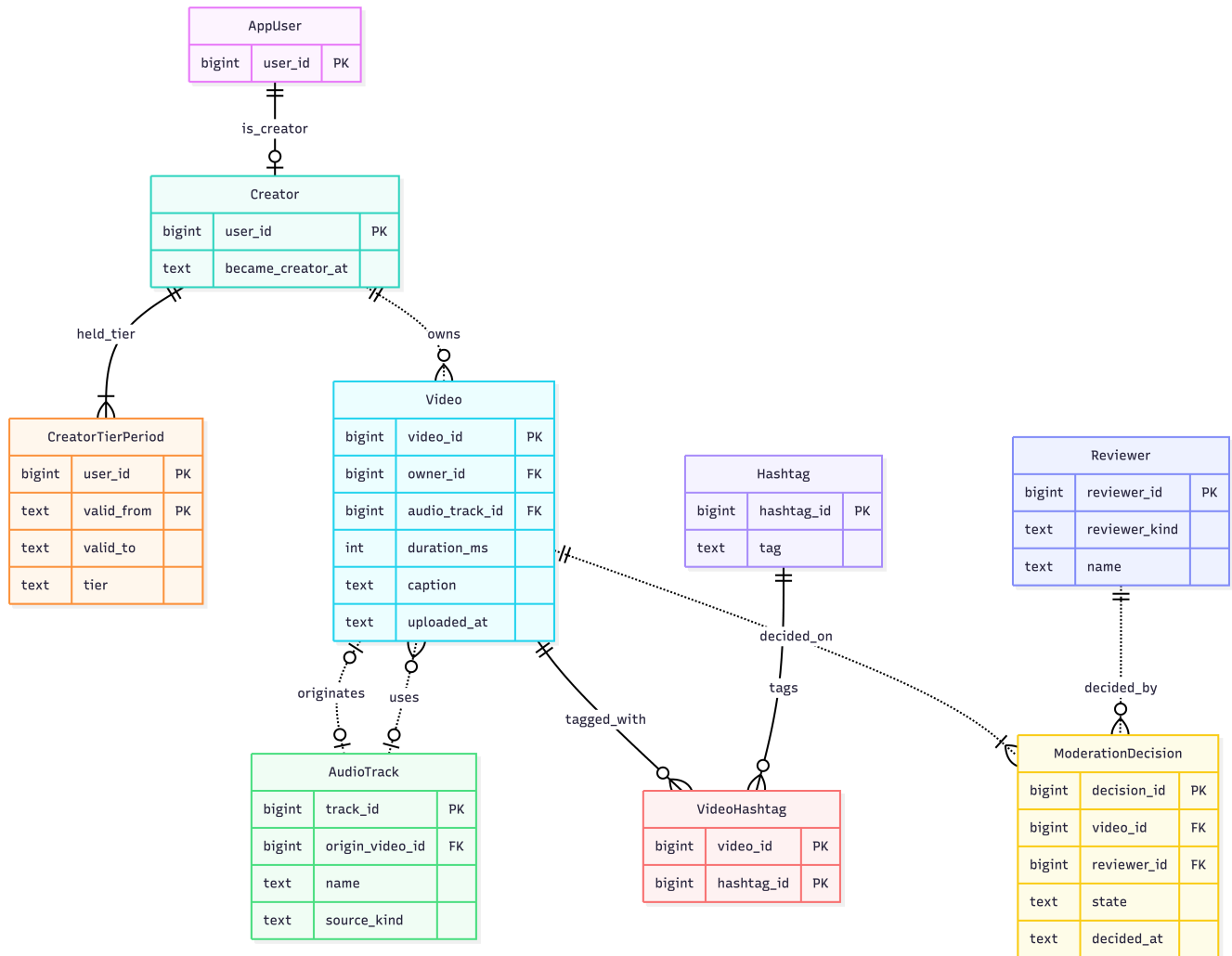


Figure 2: Content layer

Stub: AppUser (defined in the social layer).

3. Telemetry layer

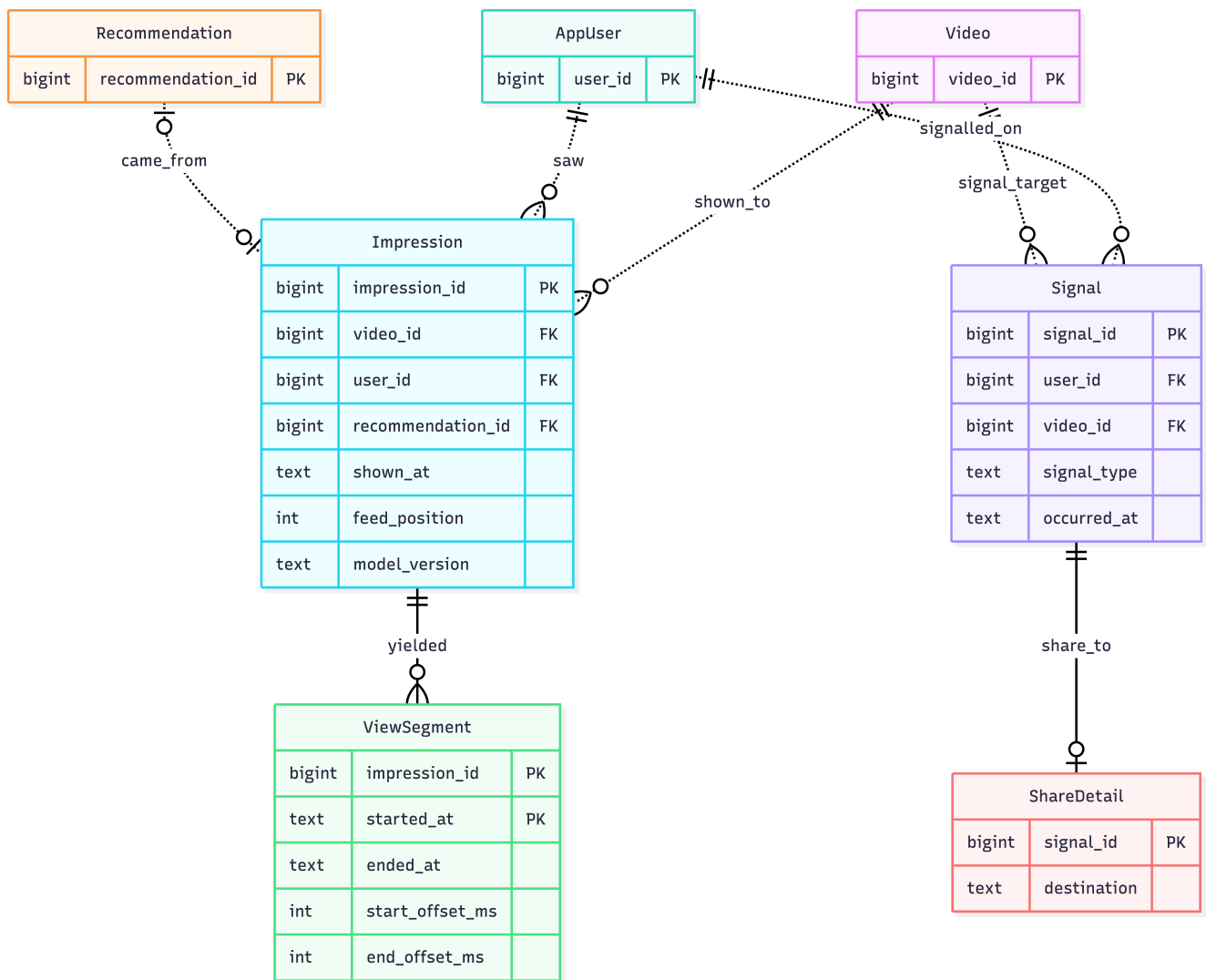


Figure 3: Telemetry layer

Stubs: Video and AppUser (social and content layers), Recommendation (agent layer).

4. Agent layer

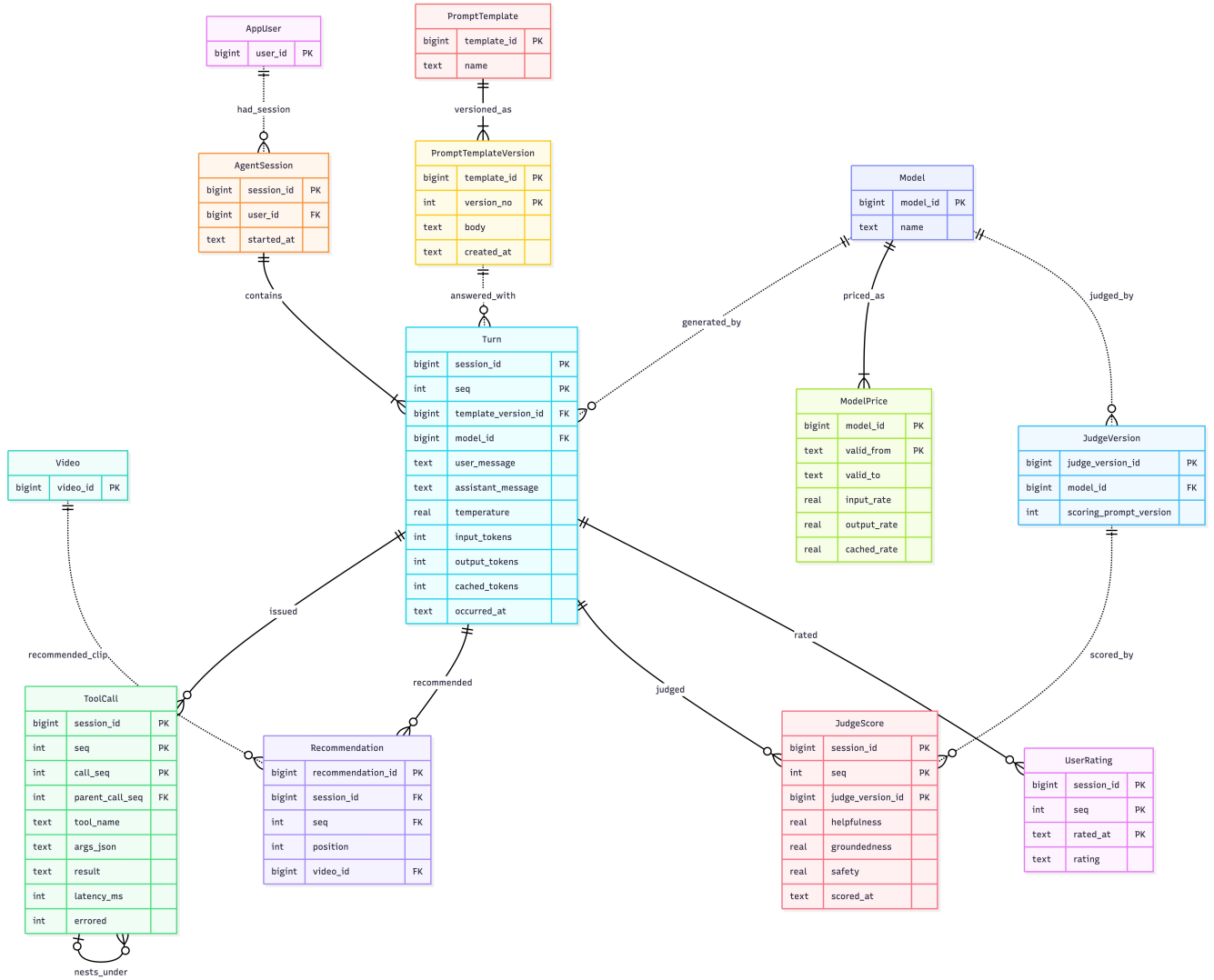


Figure 4: Agent layer

Subs: AppUser (social layer), Video (content layer).

5. Overview

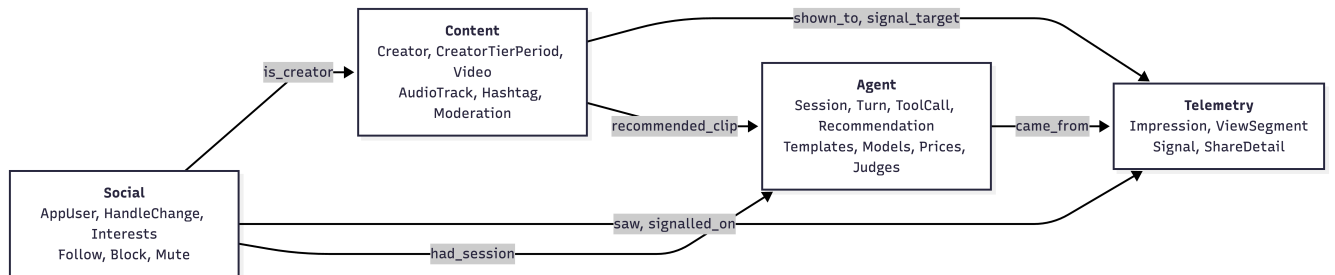


Figure 5: Overview

A layer map rather than part of the ER model, so it is not in crow's foot notation. It shows the six relationships that cross layer boundaries.

B.2 — Temporal register

Entity	Overwrite	Intervals	Event log	Justification
Creator monetisation tier		✓		If Finance asks which tier a creator was in last March, we need intervals to answer it. A tier is the same for a few months at a time, so one row per interval is enough.
Video moderation state			✓	Each row represents a decision taken at a certain time on the state of the video. We can find the video's current state by checking the latest decision, this way the two-write problem doesn't come up.
Inferred user interests		✓		The weekly refresh shouldn't destroy March's confidence score. Intervals let a refresh close the old row and open a new one.
Likes and retractions			✓	We keep a log of retractions and likes by a user with its timing since these acts can take place multiple times. A retraction is also a signal in its own right, not just the absence of a like.
Prompt template text		✓		Every previous response should stay linked to the exact text used at that time. Each version is stored as its own row with a validity window.
Model token pricing		✓		A price change should not change last quarter's costs, so cost is worked out against the rate that applied at the turn's timestamp.
Handle			✓	The twice-a-year rule counts past changes, so every change is kept with its timestamp.
Follow / Block / Mute		✓		Each of these holds for a stretch of time and is asked about as of a given moment. When a block ends a follow, we record the follow as ended rather than deleting it.
Interest suppression			✓	A suppression has no end date. Recording it as an act keeps it outside the table the weekly refresh rewrites.
Display name, phone, email	✓			Nobody asks what a display name was in March. Keeping old phone numbers and emails would also mean more copies to remove when a user deletes their account.
Declared interests	✓			The brief doesn't require what a user declared at any given time, so we can just rewrite it whenever the user changes their choices.
Impressions, view segments, signals			✓	These are occurrences rather than states. They are written once and never changed.

B.3 — The three hardest decisions

1. Likes as an event log, not a row we update

Built: likes and retractions go into one append-only stream. A like is a row and a retraction is a row.

Rejected: a Like row with an `ended_at` column, updated on retraction, plus a separate negative-signal row written at the same time.

Why: the rejected version writes one fact in two places. If the process dies between the update and the insert, we get a like marked as ended with no retraction recorded. That is the same failure 2.7 describes for moderation. It also loses repeated cycles, because a user who likes, retracts, likes and retracts again leaves one row that has been overwritten three times.

Cost of rejecting it: the rejected design answers “does this user like this clip” with one row lookup and a NULL test on `ended_at`. Ours has several rows per user and clip, so the answer needs the most recent one first, ordering by timestamp for a single clip or

a window function across many. We have given up a cheap read for a safe write.

2. Moderation state read from the log, not stored on the video

Built: ModerationDecision is an append-only log and there is no `current_state` column anywhere. Current state is the most recent decision.

Rejected: keeping the log and also a `current_state` column on Video, which is what the founders do.

Why: Trust and Safety want current state in one lookup, which argues for the column. But the column is a second write, and 2.7 says one of the two writes fails about once a week, leaving history saying `taken_down` while the column says `live`. Having only one write means the mismatch cannot happen at all.

Cost of rejecting it: the column gives the answer in one read. Ours has to search the decision log for the most recent row for that video every time. On 20,000 videos this is fine, but at a larger scale it would need an index on `(video_id, decided_at)`.

3. Recommendation linked to impression, not inferred

Built: a nullable `recommendation_id` on Impression. Feed impressions have NULL; impressions that came from an agent shelf carry the link.

Rejected: storing no link and joining Recommendation to Impression on user, video and a timestamp window.

Why: the brief asks whether clip 8821 at position 2 of turn 3 was watched and for how long. The inferred join breaks whenever a user sees the same clip in the feed shortly after seeing it in a shelf, because nothing says which impression came from where. The window size would need tuning and the answer would still be a guess.

Cost of rejecting it: Impression is the largest table in the design, and the new column is NULL on most of its rows, since most impressions come from the feed rather than an agent shelf. The code that writes impressions also has to know the recommendation id now. The rejected design kept the two layers separate.

Deliverable C — Logical Design and Relational Algebra

C.1 — Relations

Conventions used throughout:

- Timestamps are ISO-8601 text ('2026-03-03T14:32:07Z'), one representation everywhere.
- Where a column needs a specific constraint and not just a type, the range or list of allowed values is given in the comment.
- NULL is marked on every nullable column, with a note on what a NULL means there.
- Surrogate keys are used where the natural composite would otherwise grow past two columns; the composite is kept as a UNIQUE constraint. See C.2.

Social

```
AppUser (
  user_id          : bigint          -- PK
  handle           : varchar(30)     -- normalised; UNIQUE COLLATE NOCASE
  display_name     : varchar(60)
  phone            : varchar(20) NULL -- exactly one of phone / email is non-NULL
  email            : varchar(254) NULL
  account_state    : varchar(16)     -- 'active' | 'deactivated' | 'pending_deletion'
  created_at       : text            -- ISO-8601
  deletion_requested_at : text NULL  -- non-NULL iff account_state = 'pending_deletion'
)
PK: user_id
UNIQUE: handle COLLATE NOCASE
```

```

HandleChange(
    user_id      : bigint          -- PK, FK → AppUser(user_id)
    effective_at : text            -- PK, ISO-8601
    handle       : varchar(30)    -- as normalised at the time
)
PK: (user_id, effective_at)
FK: user_id → AppUser

InterestCategory(
    category_id : bigint          -- PK
    name        : varchar(40)    -- UNIQUE, lower-case normalised
)
PK: category_id
UNIQUE: name

DeclaredInterest(
    user_id      : bigint  -- PK, FK → AppUser(user_id)
    category_id  : bigint  -- PK, FK → InterestCategory(category_id)
    declared_at  : text    -- ISO-8601
)
PK: (user_id, category_id)
FK: user_id → AppUser ; category_id → InterestCategory

InferredInterest(
    user_id      : bigint  -- PK, FK → AppUser(user_id)
    category_id  : bigint  -- PK, FK → InterestCategory(category_id)
    valid_from   : text    -- PK, ISO-8601
    valid_to     : text NULL -- NULL = still current
    confidence   : real    -- 0.0 .. 1.0
)
PK: (user_id, category_id, valid_from)
FK: user_id → AppUser ; category_id → InterestCategory

InterestSuppression(
    user_id      : bigint  -- PK, FK → AppUser(user_id)
    category_id  : bigint  -- PK, FK → InterestCategory(category_id)
    suppressed_at : text    -- ISO-8601
)
PK: (user_id, category_id)
FK: user_id → AppUser ; category_id → InterestCategory

Follow(
    follower_id : bigint          -- PK, FK → AppUser(user_id)
    followee_id : bigint          -- PK, FK → AppUser(user_id)
    started_at  : text            -- PK, ISO-8601
    ended_at    : text NULL       -- NULL = still following
    end_reason   : varchar(16) NULL -- 'unfollowed' | 'blocked' ; NULL iff ended_at IS NULL
)
PK: (follower_id, followee_id, started_at)
FK: follower_id → AppUser ; followee_id → AppUser
CHECK: follower_id <> followee_id

Block(
    blocker_id : bigint          -- PK, FK → AppUser(user_id)
    blocked_id  : bigint          -- PK, FK → AppUser(user_id)
    started_at  : text            -- PK, ISO-8601
    ended_at    : text NULL       -- NULL = block still in force
)
PK: (blocker_id, blocked_id, started_at)

```

```
FK: blocker_id → AppUser ; blocked_id → AppUser
CHECK: blocker_id <> blocked_id
```

```
Mute(
  muter_id      : bigint      -- PK, FK → AppUser(user_id)
  muted_id      : bigint      -- PK, FK → AppUser(user_id)
  started_at    : text        -- PK, ISO-8601
  ended_at      : text NULL   -- NULL = mute still in force
)
PK: (muter_id, muted_id, started_at)
FK: muter_id → AppUser ; muted_id → AppUser
CHECK: muter_id <> muted_id
```

Content

```
Creator(
  user_id          : bigint  -- PK, FK → AppUser(user_id)
  became_creator_at : text    -- ISO-8601
)
PK: user_id
FK: user_id → AppUser

CreatorTierPeriod(
  user_id      : bigint      -- PK, FK → Creator(user_id)
  valid_from   : text        -- PK, ISO-8601
  valid_to     : text NULL   -- NULL = current tier
  tier          : varchar(16) -- 'none' | 'basic' | 'partner'
)
PK: (user_id, valid_from)
FK: user_id → Creator

Video(
  video_id      : bigint      -- PK
  owner_id      : bigint      -- FK → Creator(user_id)
  audio_track_id : bigint NULL -- FK → AudioTrack(track_id); NULL = no track
  duration_ms    : integer     -- 20000 .. 90000
  caption        : varchar(2200) NULL -- NULL = creator left it blank
  uploaded_at    : text        -- ISO-8601
)
PK: video_id
FK: owner_id → Creator ; audio_track_id → AudioTrack (DEFERRABLE)

AudioTrack(
  track_id          : bigint      -- PK
  origin_video_id   : bigint NULL -- FK → Video(video_id); NULL if licensed or origin removed
  name              : varchar(120)
  source_kind       : varchar(10) -- 'original' | 'licensed'
)
PK: track_id
FK: origin_video_id → Video (DEFERRABLE, ON DELETE SET NULL)
CHECK: source_kind = 'licensed' → origin_video_id IS NULL

Hashtag(
  hashtag_id : bigint      -- PK
  tag         : varchar(60) -- UNIQUE, lower-case, no leading '#'
)
PK: hashtag_id
UNIQUE: tag

VideoHashtag(
```

```

    video_id    : bigint    -- PK, FK → Video(video_id)
    hashtag_id  : bigint    -- PK, FK → Hashtag(hashtag_id)
)
PK: (video_id, hashtag_id)
FK: video_id → Video ; hashtag_id → Hashtag

Reviewer(
    reviewer_id    : bigint        -- PK
    reviewer_kind  : varchar(10) -- 'human' | 'classifier'
    name           : varchar(60) -- reviewer name or classifier version string
)
PK: reviewer_id

ModerationDecision(
    decision_id : bigint        -- PK
    video_id    : bigint        -- FK → Video(video_id)
    reviewer_id : bigint        -- FK → Reviewer(reviewer_id)
    state       : varchar(16) -- 'pending' | 'live' | 'age_restricted' | 'demoted' | 'taken_down'
    decided_at  : text          -- ISO-8601
)
PK: decision_id
FK: video_id → Video ; reviewer_id → Reviewer
UNIQUE: (video_id, decided_at)

```

Telemetry

```

Impression(
    impression_id    : bigint        -- PK
    video_id         : bigint        -- FK → Video(video_id)
    user_id          : bigint        -- FK → AppUser(user_id)
    recommendation_id : bigint NULL  -- FK → Recommendation(recommendation_id)
                                -- NULL = came from the feed, not an agent shelf
    shown_at         : text          -- ISO-8601
    feed_position     : integer       -- >= 0
    model_version     : varchar(20)  -- ranker version that placed the clip
)
PK: impression_id
FK: video_id → Video ; user_id → AppUser ; recommendation_id → Recommendation

ViewSegment(
    impression_id    : bigint    -- PK, FK → Impression(impression_id)
    started_at       : text      -- PK, ISO-8601
    ended_at         : text      -- ISO-8601; > started_at
    start_offset_ms  : integer    -- >= 0, position within the clip
    end_offset_ms    : integer    -- > start_offset_ms, <= Video.duration_ms
)
PK: (impression_id, started_at)
FK: impression_id → Impression

Signal(
    signal_id    : bigint        -- PK
    user_id      : bigint        -- FK → AppUser(user_id)
    video_id     : bigint        -- FK → Video(video_id)
    signal_type  : varchar(20) -- 'like' | 'like_retracted' | 'save' | 'share' | 'comment'
                                -- | 'follow_from_feed' | 'not_interested' | 'report'
    occurred_at  : text          -- ISO-8601
)
PK: signal_id
FK: user_id → AppUser ; video_id → Video

```

```

ShareDetail(
    signal_id    : bigint        -- PK, FK → Signal(signal_id)
    destination  : varchar(20) -- 'whatsapp' | 'instagram' | 'copied_link'
)
PK: signal_id
FK: signal_id → Signal

```

Agent

```

AgentSession(
    session_id : bigint -- PK
    user_id    : bigint -- FK → AppUser(user_id)
    started_at : text    -- ISO-8601
)
PK: session_id
FK: user_id → AppUser

Turn(
    turn_id          : bigint        -- PK (surrogate)
    session_id       : bigint        -- FK → AgentSession(session_id)
    seq              : integer       -- >= 1, position within the session
    template_version_id : bigint NULL -- FK → PromptTemplateVersion; NULL = no assistant message
    model_id         : bigint NULL   -- FK → Model(model_id); NULL = no assistant message
    user_message     : text
    assistant_message : text NULL    -- NULL = the turn errored before a reply
    temperature      : real NULL    -- 0.0 .. 2.0; NULL = no assistant message
    input_tokens     : integer NULL  -- >= 0
    output_tokens    : integer NULL  -- >= 0
    cached_tokens   : integer NULL  -- >= 0
    occurred_at      : text          -- ISO-8601
)
PK: turn_id
UNIQUE: (session_id, seq)
FK: session_id → AgentSession ; template_version_id → PromptTemplateVersion ; model_id → Model

ToolCall(
    call_id      : bigint        -- PK (surrogate)
    turn_id      : bigint        -- FK → Turn(turn_id)
    call_seq     : integer       -- >= 1, order of issue within the turn
    parent_call_id : bigint NULL -- FK → ToolCall(call_id); NULL = top-level call
    tool_name     : varchar(40) -- 'search_videos' | 'get_user_history' | 'fetch_trending_audio'
    args_json    : text          -- JSON; arguments differ per tool
    result       : text NULL     -- NULL = the call errored
    latency_ms   : integer       -- >= 0
    errored      : integer       -- 0 | 1
)
PK: call_id
UNIQUE: (turn_id, call_seq)
FK: turn_id → Turn ; parent_call_id → ToolCall

Recommendation(
    recommendation_id : bigint -- PK (surrogate)
    turn_id           : bigint -- FK → Turn(turn_id)
    position          : integer -- >= 1, position in the returned shelf
    video_id          : bigint -- FK → Video(video_id)
)
PK: recommendation_id
UNIQUE: (turn_id, position)

```

```

FK: turn_id → Turn ; video_id → Video

PromptTemplate(
    template_id : bigint      -- PK
    name        : varchar(60) -- UNIQUE
)
PK: template_id
UNIQUE: name

PromptTemplateVersion(
    template_version_id : bigint      -- PK (surrogate)
    template_id          : bigint      -- FK → PromptTemplate(template_id)
    version_no           : integer     -- >= 1
    body                 : text        -- the exact text used; never updated
    created_at           : text        -- ISO-8601
)
PK: template_version_id
UNIQUE: (template_id, version_no)
FK: template_id → PromptTemplate

Model(
    model_id : bigint      -- PK
    name      : varchar(60) -- UNIQUE, e.g. 'gpt-4o-mini', 'llama-3.1-70b'
)
PK: model_id
UNIQUE: name

ModelPrice(
    model_id      : bigint      -- PK, FK → Model(model_id)
    valid_from    : text        -- PK, ISO-8601
    valid_to      : text NULL   -- NULL = price still in force
    input_rate    : real        -- >= 0, currency per 1K tokens
    output_rate   : real        -- >= 0
    cached_rate  : real        -- >= 0
)
PK: (model_id, valid_from)
FK: model_id → Model

JudgeVersion(
    judge_version_id : bigint      -- PK
    model_id          : bigint      -- FK → Model(model_id)
    scoring_prompt_version : integer -- >= 1
)
PK: judge_version_id
UNIQUE: (model_id, scoring_prompt_version)
FK: model_id → Model

JudgeScore(
    turn_id          : bigint      -- PK, FK → Turn(turn_id)
    judge_version_id : bigint      -- PK, FK → JudgeVersion(judge_version_id)
    helpfulness      : real        -- 1.0 .. 5.0
    groundedness     : real        -- 1.0 .. 5.0
    safety           : real        -- 1.0 .. 5.0
    scored_at        : text        -- ISO-8601
)
PK: (turn_id, judge_version_id)
FK: turn_id → Turn ; judge_version_id → JudgeVersion

UserRating(
    turn_id : bigint      -- PK, FK → Turn(turn_id)

```

```

    rated_at : text          -- PK, ISO-8601
    rating   : varchar(5) -- 'up' | 'down'
)
PK: (turn_id, rated_at)
FK: turn_id → Turn

```

C.2 — Mapping decision log

ER construct	Mapping applied	Alternative rejected	Why
Video → Hashtag (M:N)	Junction table VideoHashtag, plus a Hashtag table holding the normalised tag	Keep hashtags inside the caption text, as the current system does	The brief says tags are typed into captions in no consistent case. A junction lets us look up every video for a tag without scanning and parsing 20,000 captions.
AppUser → InterestCategory (three separate M:N)	Three junction tables: DeclaredInterest, InferredInterest, InterestSuppression	One junction with a <code>source</code> column marking declared, inferred or suppressed	The weekly refresh rewrites the inferred rows, and it shouldn't touch the user's own choices or suppressions. Keeping them in separate tables means the refresh cannot reach them.
Turn → Video (M:N with position)	Junction table Recommendation carrying position, given a surrogate key	Composite key (<code>turn_id</code> , <code>position</code>) with no surrogate	Impression points back at the recommendation it came from, and a foreign key has to carry every column of the key it references. With the surrogate, Impression needs one nullable foreign key column instead of two, on the largest table in the design.
Turn → JudgeVersion (M:N with scores)	Junction table JudgeScore keyed (<code>turn_id</code> , <code>judge_version_id</code>)	Judge columns on Turn	Turn would be over 90% NULL on those columns, and a second judge version scoring the same turn would have nowhere to go.
Turn (weak on AgentSession)	Surrogate <code>turn_id</code> , with (<code>session_id</code> , <code>seq</code>) kept as a UNIQUE constraint	Natural composite (<code>session_id</code> , <code>seq</code>) as the primary key	The composite inherits downward: ToolCall would be three columns, and anything below it four. Every foreign key and join would carry all of them. The surrogate stops the chain, and the UNIQUE still guarantees that turn numbers are unique within a session.
ToolCall, Recommendation, PromptTemplateVersion (weak)	Surrogate key, with the natural composite kept as a UNIQUE constraint	Natural composite as primary key	Same reason as Turn. The rule used throughout: a surrogate where the natural composite would reach three columns, the composite kept where it stops at two.
ViewSegment (weak on Impression)	Natural composite (<code>impression_id</code> , <code>started_at</code>) kept as primary key	Surrogate <code>segment_id</code>	The key is only two columns and no table sits below ViewSegment, so a surrogate would gain nothing. The composite also shows in the key itself that a segment belongs to an impression.
ToolCall nesting (any depth)	Each call points at its parent through <code>parent_call_id</code> , NULL for a top-level call	A fixed set of columns for each level, or a path string like '1.2.4'	A call can contain calls to any depth, so a fixed set of columns cannot hold it. A path string would have to be parsed and cannot be a foreign key.
Creator (generalisation of AppUser)	Separate table sharing AppUser's key	<code>is_creator</code> flag on AppUser	Tier history and creator-only attributes belong to Creator, not to every user. <code>Video.owner_id</code> points at Creator, so the database itself stops a non-creator owning a video.

ER construct	Mapping applied	Alternative rejected	Why
Creator tier (temporal)	CreatorTierPeriod, one row per interval, valid_to NULL for the current tier	tier column on Creator	A column only tells us the tier now, and Finance asks about last March.
Model pricing (temporal)	ModelPrice, one row per interval	input_rate / output_rate columns on Model	Cost is worked out against the rate that applied at the turn's timestamp. Rates kept on Model would change last quarter's reported costs as soon as a vendor changed price.
Prompt template text (temporal)	PromptTemplateVersion, one row per version, and a Turn points at a version	Text column on PromptTemplate that gets edited	Templates are edited several times a week, and every past turn should stay linked to the text used at that time.
Inferred interest confidence (temporal)	Validity intervals on InferredInterest	Overwrite the score each week	We need the score as it was at a past moment to build training rows for events at that moment.
Moderation state (temporal)	ModerationDecision, append-only, with current state read as the latest decision	The log plus a current_state column on Video	The rejected version writes one fact twice, and 2.7 says one of the two writes fails about once a week. A single write cannot go half way.
Likes and retractions	One append-only Signal stream, with current state read as the latest event	A Like row with ended_at, plus a separate retraction row	The same two-write problem as moderation, and a user who likes and retracts repeatedly would keep overwriting the same row.
Explicit signals (eight kinds)	One Signal table with a signal_type column	Eight tables, one per signal kind	All eight would have almost the same columns. The cost is that signal_type now needs a CHECK.
Share destination	Child table ShareDetail keyed on signal_id	destination column on Signal	Only shares have a destination, so the column would be NULL for the other seven signal types.
Video ↔ AudioTrack (two relationships)	Two foreign keys: Video.audio_track_id for the sound a clip uses, AudioTrack.origin_video_id for the clip a sound came from	One relationship carrying both meanings	These are different facts. Clip 500 may use track 12 while track 12 came from clip 88. The two keys point at each other, so both are declared DEFERRABLE and are checked at COMMIT.
Handle history	HandleChange, one row per change	handle column on AppUser alone	The twice-a-year rule counts past changes, so they have to be stored. The current handle stays on AppUser.
Follow / Block / Mute (temporal, self-referencing)	One table each, keyed (from, to, started_at), with ended_at	Delete the row when the relationship ends	Following A12, a block ends a follow rather than deleting it, so data for March still shows a follow that existed in March. started_at sits in the key so the same pair can follow, unfollow and follow again.

C.3 — Relational algebra

Notation: σ selection, π projection, $\bowtie$ natural or theta join, $\bowtie_{\text{left}}$ left outer join, $\triangleright$ anti-join, γ grouping with aggregation, ρ rename.

F3 — videos with no audio track

The question is the set of clips whose `audio_track_id` matches no track. Written directly as a selection on the NULL:

```
 $\pi_{\{\text{video\_id}, \text{caption}\}} ( \sigma_{\{\text{audio\_track\_id IS NULL}\}} (\text{Video}) )$ 
```

Written as an anti-join, which is what the NOT EXISTS form expresses:

```
 $\pi_{\{\text{video\_id}, \text{caption}\}} ( \text{Video} \triangleright_{\{\text{Video.audio\_track\_id} = \text{AudioTrack.track\_id}\}} \text{AudioTrack} )$ 
```

Both give 3,018 rows. The NOT IN form has no algebraic expression, because relational algebra is defined over sets with two-valued logic while SQL's NOT IN evaluates under three-valued logic.

F2 — watch hours and completion rate per creator, live clips only

Current moderation state, as the decision with the latest timestamp for each video:

```
Latest  $\leftarrow \gamma_{\{\text{video\_id}; \text{MAX}(\text{decided\_at}) \rightarrow \text{latest}\}} (\text{ModerationDecision})$   
Current  $\leftarrow \pi_{\{\text{video\_id}, \text{state}\}} ($   
           $\text{ModerationDecision} \bowtie_{\{\text{video\_id} = \text{video\_id} \wedge \text{decided\_at} = \text{latest}\}} \text{Latest} )$ 
```

Live clips, and the impressions against them aggregated to one row each:

```
Live  $\leftarrow \pi_{\{\text{video\_id}, \text{owner\_id}, \text{duration\_ms}\}} ($   
           $\text{Video} \bowtie_{\{\text{state} = \text{'live'}\}} (\text{Current}) )$   
  
PerImp  $\leftarrow \gamma_{\{\text{impression\_id}, \text{owner\_id};$   
           $\text{SUM}(\text{end\_offset\_ms} - \text{start\_offset\_ms}) \rightarrow \text{watch\_ms},$   
           $\text{MAX}(\text{end\_offset\_ms} \geq 0.95 \times \text{duration\_ms}) \rightarrow \text{completed}\}} ($   
           $(\text{Impression} \bowtie \text{Live}) \bowtie \text{ViewSegment} )$ 
```

Then the outer join that keeps every creator:

```
F2  $\leftarrow \gamma_{\{\text{user\_id};$   
           $\text{COUNT}(\text{impression\_id}) \rightarrow \text{impressions},$   
           $\text{SUM}(\text{watch\_ms}) \rightarrow \text{watch\_hours},$   
           $\text{AVG}(\text{completed}) \rightarrow \text{mean\_completion\_rate}\}} ($   
           $\text{Creator} \bowtie_{\{\text{Creator.user\_id} = \text{PerImp.owner\_id}\}} \text{PerImp} )$ 
```

$\bowtie$ against ViewSegment keeps impressions that never became views; $\bowtie$ against PerImp keeps creators with no live clips and creators never watched.

F12 — recommended clips that were watched to completion

```
RecImp  $\leftarrow \sigma_{\{\text{recommendation\_id IS NOT NULL}\}} (\text{Impression})$   
  
Completed  $\leftarrow \sigma_{\{\text{max\_off} \geq 0.95 \times \text{duration\_ms}\}} ($   
           $\gamma_{\{\text{impression\_id}, \text{recommendation\_id}, \text{duration\_ms};$   
           $\text{SUM}(\text{end\_offset\_ms} - \text{start\_offset\_ms}) \rightarrow \text{watch\_ms},$   
           $\text{MAX}(\text{end\_offset\_ms}) \rightarrow \text{max\_off}\}} ($   
           $\text{RecImp} \bowtie \text{ViewSegment} \bowtie \text{Video} ) )$   
  
F12  $\leftarrow \pi_{\{\text{session\_id}, \text{seq}, \text{position}, \text{video\_id}, \text{watch\_ms}, \text{shown\_at}\}} ($   
           $\text{Completed} \bowtie \text{Recommendation} \bowtie \text{Turn} \bowtie \text{Impression} )$ 
```

The rewrite: pushing a selection below a join

Taking F12. The unoptimised form applies the selection after joining:

Before

```
 $\sigma_{\{\text{recommendation\_id IS NOT NULL}\}} ( \text{Impression} \bowtie \text{ViewSegment} \bowtie \text{Video} )$ 
```

After

```
 $( \sigma_{\{\text{recommendation\_id IS NOT NULL}\}} (\text{Impression}) ) \bowtie \text{ViewSegment} \bowtie \text{Video}$ 
```

Why this is equivalence-preserving. The predicate mentions only attributes of Impression, so it commutes with the joins: selecting then joining and joining then selecting produce the same set of tuples. This is the standard selection-pushdown rule, $\sigma_p(R \bowtie S) \equiv \sigma_p(R) \bowtie S$ when p refers only to attributes of R .

Why the rewritten form is expected to be cheaper. Impression holds 300,000 rows and only 3,009 of them carry a recommendation. The “before” form joins all 300,000 against 106,122 view segments and then discards 99% of the result. The “after” form reduces the left input to 3,009 rows before any join runs.

I checked both forms with `EXPLAIN QUERY PLAN` and they produce the same plan, and their runtimes are within noise of each other over repeated runs. SQLite’s optimiser already performs this pushdown, because the predicate is a simple test on a base table.

Deliverable D — Normalisation

The relations in C.1 were written after considering normalisation, so they are already in BCNF and there would be nothing to decompose. The four relations below are the first-pass designs that did not satisfy normalisation. Each analysis shows where the relation fails and decomposes it.

D.1 Relation 1 — ImpressionSegment (telemetry)

```
RELATION: ImpressionSegment(impression_id, video_id, user_id, shown_at,  
                             feed_position, model_version, started_at,  
                             ended_at, start_offset_ms, end_offset_ms,  
                             duration_ms)
```

One row represents a continuous stretch of playback.

Functional Dependencies:

- **F1: $\text{impression_id} \rightarrow \text{video_id}, \text{user_id}, \text{shown_at}, \text{feed_position}, \text{model_version}$** Because one impression is one clip entering one user’s viewport at one instant, ranked into one feed position by one model version. All of those are fixed the moment the impression is fixed.
- **F2: $\text{impression_id}, \text{started_at} \rightarrow \text{ended_at}, \text{start_offset_ms}, \text{end_offset_ms}$** Because segments of one impression cannot overlap in time, so once an impression and its start time are fixed, the rest of the attributes also get fixed.
- **F3: $\text{video_id} \rightarrow \text{duration_ms}$** Because a clip has one length.

Closures

- $\{\text{impression_id}, \text{started_at}\}^+ = \text{start with impression_id, started_at; F1 adds video_id, user_id, shown_at, feed_position, model_version; F2 adds ended_at, start_offset_ms, end_offset_ms; F3 adds duration_ms. All attributes.}$
- $\{\text{impression_id}\}^+ = \text{impression_id, video_id, user_id, shown_at, feed_position, model_version, duration_ms. Missing the four segment attributes, so not a superkey.}$
- $\{\text{video_id}\}^+ = \text{video_id, duration_ms. Not a superkey.}$

Candidate keys

`impression_id` and `started_at` never appear on the right of any FD, so both are in every candidate key. Their closure is already all attributes, so $\{\text{impression_id}, \text{started_at}\}$ is the only candidate key.

Prime attributes: `impression_id`, `started_at`.

Verdict: not 2NF.

- F1's left side is a proper subset of the key, so the impression attributes depend on part of the key rather than all of it. That is a partial dependency and breaks 2NF.
- F3 reaches `duration_ms` through `video_id`, which is not a key, and `duration_ms` is not prime. That is a transitive dependency and breaks 3NF.
- Both left sides are non-superkeys, so both break BCNF.

Decomposition

```
R1a(impression_id, video_id, user_id, shown_at, feed_position, model_version)
R1b(impression_id, started_at, ended_at, start_offset_ms, end_offset_ms)
R1c(video_id, duration_ms)
```

Lossless

- $R1a \cap R1b = \{impression_id\}$, which is a key of R1a.
- $(R1a \bowtie R1b) \cap R1c = \{video_id\}$, which is a key of R1c.

Dependency preserving: F1 in R1a, F2 in R1b, F3 in R1c.

R1a is Impression, R1b is ViewSegment, R1c is the relevant part of Video.

D.1 Relation 2 — CreatorTierFull (temporal)

```
RELATION: CreatorTierFull(user_id, handle, became_creator_at,
                          valid_from, valid_to, tier)
```

One row is one creator's tier over one interval, with the creator's handle repeated on every interval.

Functional Dependencies:

- **F1: `user_id` → `handle`, `became_creator_at`** Because a user has one current handle and became a creator once.
- **F2: `handle` → `user_id`** Because handles are unique, so a handle picks out exactly one account.
- **F3: `user_id`, `valid_from` → `valid_to`, `tier`** Because a creator is in one tier over any one interval, and intervals for a creator do not overlap.

Closures

- $\{user_id, valid_from\}^+ = user_id, valid_from$, F1 adds `handle`, `became_creator_at`, F3 adds `valid_to`, `tier`. All attributes.
- $\{handle, valid_from\}^+ = handle, valid_from$, F2 adds `user_id`, then F1 adds `became_creator_at` and F3 adds `valid_to`, `tier`. All attributes.
- $\{user_id\}^+ = user_id, handle, became_creator_at$. Not a superkey.
- $\{handle\}^+ = handle, user_id, became_creator_at$. Not a superkey.

Candidate keys

`valid_from` never appears on the right of any FD, so it is in every candidate key. `became_creator_at`, `valid_to` and `tier` never appear on the left, so they are in no candidate key. That leaves `user_id` and `handle`. $\{valid_from\}^+$ is just `valid_from`, so it needs one of the two to become part of a candidate key, and each of them works.

Two candidate keys: $\{user_id, valid_from\}$ and $\{handle, valid_from\}$.

Prime attributes: `user_id`, `handle`, `valid_from`.

Verdict: not BCNF.

- F1: $\{user_id\}^+$ is not all attributes, so `user_id` is not a superkey. BCNF violated. It also breaks 3NF, because `became_creator_at` is not prime.
- F2: `handle` is not a superkey either, so BCNF is violated. But `user_id` is prime, so 3NF is satisfied by this FD.

Decomposition

```
R2a(user_id, handle, became_creator_at)
R2b(user_id, valid_from, valid_to, tier)
```

Lossless: $R2a \bowtie R2b = \{user_id\}$, which is a key of $R2a$.

Dependency preserving: $F1$ and $F2$ in $R2a$, $F3$ in $R2b$.

$R2a$ is BCNF: its candidate keys are $\{user_id\}$ and $\{handle\}$, since each determines the other. $F1$'s left side $user_id$ is a superkey and $F2$'s left side $handle$ is a superkey, so no FD violates BCNF.

$R2b$ is $CreatorTierPeriod$.

D.1 Relation 3 — JudgeScoreFull (agent layer)

RELATION: JudgeScoreFull($turn_id$, $judge_version_id$, $model_id$, $model_name$,
 $scoring_prompt_version$, $helpfulness$,
 $groundedness$, $safety$, $scored_at$)

One row is one judge version's scoring of one turn, with the judge's model details repeated on every score.

Functional Dependencies:

- **$F1$:** $turn_id, judge_version_id \rightarrow helpfulness, groundedness, safety, scored_at$ Because a given judge version scores a given turn once.
- **$F2$:** $judge_version_id \rightarrow model_id, scoring_prompt_version$ Because a judge version is defined as one model paired with one scoring-prompt version.
- **$F3$:** $model_id, scoring_prompt_version \rightarrow judge_version_id$ Because that pairing is unique: the same model with the same scoring prompt is the same judge version, not a second one.
- **$F4$:** $model_id \rightarrow model_name$ Because a model has one name.

Closures

- $\{turn_id, judge_version_id\}^+ =$ those two, $F2$ adds $model_id, scoring_prompt_version$, $F4$ adds $model_name$, $F1$ adds the three scores and $scored_at$. All attributes.
- $\{turn_id, model_id, scoring_prompt_version\}^+ =$ those three, $F3$ adds $judge_version_id$, $F4$ adds $model_name$, $F1$ adds the scores and $scored_at$. All attributes.
- $\{judge_version_id\}^+ = judge_version_id, model_id, scoring_prompt_version, model_name$. Not a superkey.
- $\{model_id\}^+ = model_id, model_name$. Not a superkey.

Candidate keys

$turn_id$ never appears on the right, so it is in every candidate key. The scores, $scored_at$ and $model_name$ never appear on the left, so they are in none. Adding $judge_version_id$ closes it, and so does adding $model_id, scoring_prompt_version$.

Two candidate keys: $\{turn_id, judge_version_id\}$ and $\{turn_id, model_id, scoring_prompt_version\}$.

Prime attributes: $turn_id, judge_version_id, model_id, scoring_prompt_version$.

Verdict: not BCNF.

- $F2$: $judge_version_id$ is not a superkey, so BCNF is violated. Both $model_id$ and $scoring_prompt_version$ are prime, so 3NF is satisfied by this FD.
- $F4$: $model_id$ is not a superkey and $model_name$ is not prime, so this breaks both 3NF and BCNF.

Decomposition

$R3a(turn_id, judge_version_id, helpfulness, groundedness, safety, scored_at)$
 $R3b(judge_version_id, model_id, scoring_prompt_version)$
 $R3c(model_id, model_name)$

Lossless

- $R3a \bowtie R3b = \{judge_version_id\}$, which is a key of $R3b$.
- $(R3a \bowtie R3b) \bowtie R3c = \{model_id\}$, which is a key of $R3c$.

Dependency preserving: $F1$ in $R3a$, $F2$ and $F3$ in $R3b$, $F4$ in $R3c$.

$R3a$ is JudgeScore, $R3b$ is JudgeVersion, $R3c$ is Model.

D.1 Relation 4 — VideoTagged (content)

RELATION: VideoTagged(video_id, owner_id, owner_handle, duration_ms, caption, uploaded_at, tag)

One row is one tag on one video, which is what you get if tags are read out of the caption text and the video's details are carried alongside.

Functional Dependencies:

- **F1: video_id → owner_id, duration_ms, caption, uploaded_at** Because a clip is uploaded once, by one creator, with one length and one caption.
- **F2: owner_id → owner_handle** Because a user has one current handle.
- **F3: owner_handle → owner_id** Because handles are unique.

Closures

- {video_id, tag}⁺ = those two, F1 adds owner_id, duration_ms, caption, uploaded_at, F2 adds owner_handle. All attributes.
- {video_id}⁺ = video_id, owner_id, duration_ms, caption, uploaded_at, owner_handle. Missing tag, so not a superkey.
- {owner_id}⁺ = owner_id, owner_handle. Not a superkey.

Candidate keys

video_id and tag never appear on the right of any FD, so both are in every candidate key. The closure computed above shows those two alone already give every attribute, so no smaller or larger key is possible. {video_id, tag} is the only candidate key.

Prime attributes: video_id, tag.

Verdict: not 2NF.

- F1's left side is a proper subset of the key, so every video attribute is repeated once per tag. Partial dependency, breaks 2NF.
- F2 reaches owner_handle through owner_id, which is not a key, and owner_handle is not prime. Transitive, breaks 3NF.
- Both left sides are non-superkeys, so both break BCNF.

Decomposition

R4a(video_id, tag)

R4b(video_id, owner_id, duration_ms, caption, uploaded_at)

R4c(owner_id, owner_handle)

Lossless

- $R4a \cap R4b = \{video_id\}$, which is a key of R4b.
- $(R4a \bowtie R4b) \cap R4c = \{owner_id\}$, which is a key of R4c.

Dependency preserving: F1 in R4b, F2 and F3 in R4c.

R4a is VideoHashtag (with the tag replaced by a foreign key to Hashtag), R4b is Video, R4c is the relevant part of AppUser.

Where 3NF was kept instead of BCNF

All four decompositions above reach BCNF, so no such case arises. In R2, $handle \rightarrow user_id$ violates BCNF while satisfying 3NF, and in R3 $judge_version_id \rightarrow model_id, scoring_prompt_version$ does the same. In both cases the decomposition to BCNF also preserved every dependency. The choice between 3NF and BCNF only arises when the two conflict, and here they did not. Where a BCNF decomposition loses a dependency, stopping at 3NF is correct, because a lost dependency has to be enforced by application code across a join, which can cause violations later on.

D.2 — The denormalisation I did not do

The column considered: a `like_count` on Video, incremented on a like and decremented on a retraction.

The query it would accelerate. Every feed render and every profile page shows a like count per clip. Without the column that count is derived from the Signal stream: for each user who has signalled on the clip, find their most recent like or retraction, and count the ones that are likes. That is a window function over the whole signal history of the clip, run for every clip on screen. The column would replace all of it with a single integer read.

The update anomaly it would admit. The count and the signal stream are the same fact stored twice. Writing a like means one insert into Signal and one increment on Video. If the process dies between them, the count disagrees with the stream and nothing detects it. This is the failure described in 2.7.

The enforcement mechanism it would require. The Signal insert and the count update would have to go inside one transaction, so a crash rolls back both instead of leaving one applied. That only helps if the code remembers to do both writes, so on top of it an `AFTER INSERT` trigger on Signal would update the count itself: a `like` adds one, a `like_retracted` takes one away. A periodic job would then recount from the signal stream and compare against the stored counts, to catch anything the first two miss.

Why I did not do it. The count is already available without the column, by reading the signal stream, and an index on `(video_id, user_id, occurred_at)` makes that read fast enough at the volumes in this assignment. Adding the column would store the same fact in two places and need a transaction, a trigger and a reconciliation job to keep the two in agreement.

Deliverable E — Physical Implementation

E.1 — Schema notes

The full DDL is in `schema.sql`. It creates 35 tables. The two pragmas are at the top of the file, and `PRAGMA foreign_keys = ON` is also set on the generator's connection, since it is per-connection rather than per-database.

Type affinity

I declared a test table with a plain `INTEGER` column and inserted the text `'hello'` into it:

```
INSERT INTO AffinityTest (n) VALUES ('hello');
SELECT n, typeof(n) FROM AffinityTest;  ->  'hello', text
```

No error. The value is stored as text in a column declared `INTEGER`. Inserting `'123'` into the same column stores the integer 123, because SQLite coerces a value that looks numeric and keeps one that does not.

The one exception is `INTEGER PRIMARY KEY`, which is an alias for the rowid and does reject text with a datatype mismatch. That covers the primary key of every table here, but no other column.

A type name in this schema is a hint, not a constraint. Every rule that holds is held by a `CHECK`, a `UNIQUE`, a `NOT NULL` or a foreign key. `duration_ms INTEGER` does not stop text being stored there; `CHECK (duration_ms BETWEEN 20000 AND 90000)` does.

STRICT tables

I did not declare the tables `STRICT`. Under `STRICT` the same insert is rejected with “cannot store TEXT value in INTEGER column”, which is the behaviour I would want. The cost is that `STRICT` accepts only `INT`, `INTEGER`, `REAL`, `TEXT`, `BLOB` and `ANY`. I checked both of the type names the sample databases use:

```
CREATE TABLE t (d DATETIME) STRICT;          ->  unknown datatype: "DATETIME"
CREATE TABLE t (v NVARCHAR(160)) STRICT;      ->  unknown datatype: "NVARCHAR(160)"
```

Adopting `STRICT` would mean writing `TEXT` for every timestamp and every bounded string, which loses the documentation value of `varchar(30)` and `varchar(2200)` in the DDL. Since those lengths are already enforced by `CHECK (length(...)) <= n`, the trade is between a type name that reads well and an engine-level guarantee. I kept the readable type names and the `CHECK` constraints.

Handle uniqueness

```
CREATE UNIQUE INDEX ux_appuser_handle ON AppUser (handle COLLATE NOCASE);
```

NOCASE folds ASCII A–Z only, so it does nothing outside that range. Tamil has no case distinction, so there are no case-variant pairs to collide and the constraint behaves as plain equality, which is correct for that script. The gap shows up elsewhere: 'CAFÉ' = 'café' COLLATE NOCASE returns 0, so two handles differing only in the case of a non-ASCII letter would both be accepted.

Unique among *active* accounts. A UNIQUE applies to the whole column, so it cannot be limited to active rows. I chose to enforce uniqueness across all accounts instead, which is stricter than the brief asks: a deactivated account keeps its handle reserved. Releasing a handle when an account is deactivated would need either a partial unique index on `account_state = 'active'`, or application code that clears the handle on deactivation. The stricter rule also avoids the case where an account inside its 30-day recovery window is restored to find its handle taken.

Timestamps

Every timestamp in the schema is ISO-8601 text, '2026-03-03T14:32:07Z'. Playhead offsets and durations are integers in milliseconds.

ISO-8601 text sorts correctly as text, so ORDER BY and BETWEEN work directly, and `date()`, `strftime()` and `julianday()` all accept it. Three of the queries in Deliverable F depend on this: the 60-second window between a like and its retraction needs `julianday(b) - julianday(a)` scaled to seconds, consecutive active days needs `date(shown_at)`, and the 7-day rolling window needs ORDER BY `julianday(...)` with RANGE BETWEEN 6 PRECEDING AND CURRENT ROW, since SQLite's RANGE frames take numeric offsets only.

The cost of text over epoch integers is size and a conversion on every date calculation. The benefit is that the data is readable without conversion, which matters when debugging a join that returns nothing.

NOT NULL audit

Table.column	What a NULL means
AppUser.phone	the user registered with a Google account instead
AppUser.email	the user registered with a phone number instead
AppUser.deletion_requested_at	does not apply: the account is not inside a deletion window
InferredInterest.valid_to	still current: this is the interval the weekly refresh last wrote
Follow.ended_at	still following
Follow.end_reason	does not apply: the follow has not ended
Block.ended_at, Mute.ended_at	still in force
CreatorTierPeriod.valid_to	this is the creator's current tier
ModelPrice.valid_to	this price is still in force
Video.audio_track_id	the creator chose no track
Video.caption	the creator left the caption blank
AudioTrack.origin_video_id	either the track is licensed, or its origin video has been removed
Impression.recommendation_id	the impression came from the feed, not an agent shelf
Turn.assistant_message	the turn errored before a reply was produced
Turn.template_version_id, model_id, temperature, input_tokens, output_tokens, cached_tokens	does not apply: there is no assistant message
ToolCall.parent_call_id	a top-level call, not nested under another
ToolCall.result	the call errored

`AudioTrack.origin_video_id` is the one column where a NULL means two different things. Either the track is licensed, so it never had an origin video, or its origin video was removed. `source_kind` separates them: a NULL origin on an original track is the second case.

ON DELETE

The brief's deletion model is soft. An account moves to `pending_deletion` and stays restorable for 30 days, so a hard `DELETE` is the exception rather than the normal path. The choices below are about what should happen if one is issued.

RESTRICT on `Impression.user_id`, `Signal.user_id`, `Follow.follower_id`, `Follow.followee_id` and `AgentSession.user_id`. Each of these carries data a consumer named in 2.6 depends on: watch history and signals for the ML team, session costs for Finance, the social graph for reconstructing past state. Deleting a user would take all of it with them. The database refuses instead, so the only way to remove a user is the soft path the brief describes.

RESTRICT on `Video.owner_id`, so a creator with videos cannot be removed, and on every foreign key into a lookup table, so a moderation state still in use cannot be deleted. Also on `ModerationDecision.reviewer_id`, so a decision keeps its attribution.

CASCADE where the child row is the user's own setting and has no meaning without them: `HandleChange`, the three interest tables, `Block`, `Mute`. Also from `Video` down to `ModerationDecision`, `VideoHashtag`, `Impression`, `Signal` and `Recommendation`, and from `AgentSession` down through `Turn` to `ToolCall`, `Recommendation`, `JudgeScore` and `UserRating`.

SET NULL in two places. `AudioTrack.origin_video_id`, so a takedown of the originating clip does not break the tens of thousands of clips borrowing the sound. And `Impression.recommendation_id`, so removing a recommendation does not remove the impression that followed it.

Closed value sets

Three sets became lookup tables and five stayed as `CHECK` constraints.

Lookup tables for moderation state, creator tier and share destination. Each set is owned outside engineering, so Trust and Safety, Finance and Growth can add values without a schema change. Each value also carries attributes a `CHECK` cannot hold, such as whether a moderation state still shows in the feed, or a tier's revenue share.

CHECK constraints for account state, source kind, reviewer kind, signal type and tool name. Adding a value to any of these needs code written: a new account state means rewriting the deletion and recovery logic, a new signal type needs the client to emit it, a new tool needs implementing. Since the set cannot change without an engineering change, there is nothing gained by making it editable data.

The cost is that SQLite has no `ALTER TABLE ... ADD CONSTRAINT`, so changing one of these sets means recreating the table. That is acceptable for sets that only change alongside code.

Creation order and the one cycle

SQLite has no `ALTER TABLE ... ADD CONSTRAINT`, so a table can only reference tables that already exist and the order in the file is fixed: lookup tables, then `AppUser` and its dependents, then `Creator`, then content, then the agent layer, then telemetry.

One pair is circular. `Video.audio_track_id` references `AudioTrack`, and `AudioTrack.origin_video_id` references `Video`. `AudioTrack` is created first with its foreign key marked `DEFERRABLE INITIALLY DEFERRED`, so the constraint is checked at `COMMIT` rather than per statement. The generator sidesteps the cycle a second way: it inserts tracks with a `NULL` origin, inserts the videos, then fills the origins in with an `UPDATE`.

E.2 — Constraint trace table

#	Rule in words	Enforced by	Where
A1	An impression is recorded whether or not it becomes a view	separate tables; <code>ViewSegment</code> rows are optional	<code>Impression</code> , <code>ViewSegment</code>
A2	No two segments of one impression share a start time	<code>PRIMARY KEY (impression_id, started_at)</code>	<code>ViewSegment</code>
A2	A segment ends after it starts, and covers a forward stretch of the clip	<code>CHECK (ended_at > started_at), CHECK (end_offset_ms > start_offset_ms)</code>	<code>ViewSegment</code>
A2	Segments of one impression do not overlap in time	not declarative — application code or a trigger	—

#	Rule in words	Enforced by	Where
A3	An impression is complete if a segment reached 95% of the clip	not declarative — defined in the queries that report completion	—
A4	Likes and retractions are append-only, never updated	not declarative — application code; no UPDATE path is provided	—
A5	One share to two destinations is two rows	PRIMARY KEY (signal_id) on ShareDetail, so one signal carries one destination	ShareDetail
A6	A turn has one user message and at most one assistant message	NOT NULL on user_message; assistant_message nullable	Turn
A6	A turn with no assistant message has no template, model, temperature or token counts	four paired CHECK ((assistant_message IS NULL) = (col IS NULL))	Turn
A7	A tool call belongs to exactly one turn	turn_id INTEGER NOT NULL REFERENCES Turn	ToolCall
A7	Nesting is a parent pointer, and a call is not its own parent	parent_call_id self-reference, CHECK (parent_call_id <> call_id)	ToolCall
A7	Calls are ordered within their turn	UNIQUE (turn_id, call_seq)	ToolCall
A8	A recommendation is not an impression, and may never become one	separate tables joined by a nullable FK	Recommendation, Impression
A9	A user may rate a turn more than once, at different times	PRIMARY KEY (turn_id, rated_at)	UserRating
A10	A judge version scores a turn at most once	PRIMARY KEY (turn_id, judge_version_id)	JudgeScore
A10	A judge version is one model with one scoring-prompt version	UNIQUE (model_id, scoring_prompt_version)	JudgeVersion
A11	A template has numbered versions and a turn points at one	UNIQUE (template_id, version_no); Turn.template_version_id FK	PromptTemplateVersion, Turn
A11	Version text is never edited after it is written	not declarative — application code or a trigger	—
A12	A follow that has ended records when and why	CHECK ((ended_at IS NULL) = (end_reason IS NULL))	Follow
A12	A block breaks follows in both directions	not declarative — application code or a trigger	—
A12	A user cannot follow, block or mute themselves	CHECK (follower_id <> followee_id) and the same on Block and Mute	Follow, Block, Mute
A13	A handle changes at most twice in any rolling 365 days	not declarative — application code; HandleChange stores the history it needs	—
A14	A track survives its origin video being removed	ON DELETE SET NULL on origin_video_id	AudioTrack
A14	A licensed track has no origin video	CHECK (source_kind = 'original' OR origin_video_id IS NULL)	AudioTrack
A15	A user with watch history, signals, follows or sessions cannot be hard-deleted	ON DELETE RESTRICT on those four foreign keys to AppUser	Impression, Signal, Follow, AgentSession
A16	A video carries zero or one audio track	single nullable audio_track_id column	Video

Rules from the brief that are not tied to a numbered assumption:

Rule in words	Enforced by	Where
A handle is unique, case-insensitively	CREATE UNIQUE INDEX ... ON AppUser (handle COLLATE NOCASE)	AppUser
A handle is unique only among active accounts	not declarative — application code	—
A handle carries no leading or trailing whitespace and is 3–30 characters	CHECK (handle = trim(handle) AND length(handle) BETWEEN 3 AND 30)	AppUser

Rule in words	Enforced by	Where
A user registers with a phone number or a Google account	CHECK (phone IS NOT NULL OR email IS NOT NULL)	AppUser
An account inside its deletion window has a deletion timestamp, and no other account does	CHECK ((account_state = 'pending_deletion') = (deletion_requested_at IS NOT NULL))	AppUser
A clip is 20 to 90 seconds	CHECK (duration_ms BETWEEN 20000 AND 90000)	Video
A caption is at most 2200 characters	CHECK (caption IS NULL OR length(caption) <= 2200)	Video
A hashtag is stored lower-case, trimmed, without a leading #	CHECK (tag = lower(trim(tag)) AND tag NOT LIKE '#%')	Hashtag
A video belongs to a creator, not to any user	owner_id references Creator rather than AppUser	Video
Only one moderation decision exists for a video at a given instant	UNIQUE (video_id, decided_at)	ModerationDecision
Moderation states, creator tiers and share destinations come from a fixed set	foreign keys to lookup tables	ModerationDecision, CreatorTierPeriod, ShareDetail
Account states, source kinds, reviewer kinds, signal types and tool names come from a fixed set	CHECK (col IN (...))	AppUser, AudioTrack, Reviewer, Signal, ToolCall
Validity intervals end after they start	CHECK (valid_to IS NULL OR valid_to > valid_from)	InferredInterest, CreatorTierPeriod, ModelPrice
Two open intervals cannot exist for the same subject	not declarative — application code or a trigger	—
An inferred interest carries a confidence between 0 and 1	CHECK (confidence BETWEEN 0 AND 1)	InferredInterest
Judge scores fall between 1 and 5	CHECK (col BETWEEN 1 AND 5) on all three	JudgeScore
Token counts, latencies and prices are not negative	CHECK (col >= 0)	Turn, ToolCall, ModelPrice
Temperature is between 0 and 2	CHECK (temperature IS NULL OR temperature BETWEEN 0 AND 2)	Turn
A tool call that errored has no result, and one that succeeded has one	CHECK ((errored = 1) = (result IS NULL))	ToolCall
Tool call arguments are valid JSON	CHECK (json_valid(args_json))	ToolCall

Seven rules cannot be expressed declaratively in SQLite: segment non-overlap, the completion threshold, the append-only discipline on signals and template versions, the block-breaks-follow rule, the rolling handle limit, active-only handle uniqueness, and non-overlapping validity intervals. Each is enforced in application code.

E.3 — Data generator

`generate_data.py`, seeded with 49 from roll number DA24B049. The parameter block sits at the top of the file so Assignment 2 can change only that:

```
SEED = 49
N_USERS = 5_000
N_VIDEOS = 20_000
N_IMPRESSIONS = 300_000
N_AGENT_SESSIONS = 2_000
SCALE = 1
```

The whole load runs inside one transaction with `executemany`. SQLite commits after every statement otherwise, and each

commit is an fsync, so 300,000 auto-committed inserts would take longer than the rest of the assignment. Inside one transaction the full load takes under ten seconds. `PRAGMA foreign_keys = ON` is set on the generator's connection as well as in `schema.sql`, since the pragma is per-connection.

Tables are loaded in dependency order. The Video and AudioTrack cycle is handled by inserting tracks with a NULL origin, inserting the videos, then filling the origins in with an UPDATE.

Rows produced

Table	Rows	Table	Rows
AppUser	5,000	AgentSession	2,000
Creator	1,250	Turn	4,931
Video	20,000	ToolCall	6,722
AudioTrack	2,000	Recommendation	14,937
Hashtag	800	JudgeScore	1,119
VideoHashtag	39,806	UserRating	157
ModerationDecision	40,825	Impression	300,000
Follow	59,232	ViewSegment	106,122
Block	2,000	Signal	7,841
Mute	2,999	ShareDetail	723

Distributions

The funnel leaks at every stage. 28% of impressions produce at least one view segment, and 9% of those produce an explicit signal. Both are drawn as a probability per impression rather than a fixed ratio, so the leak varies by clip.

Watch time is right-skewed. Each segment covers a fraction of the clip drawn from a $\text{Beta}(1.4, 2.2)$ distribution, which puts most of the mass below half the clip with a thin tail reaching the end. An impression gets one segment 78% of the time, two 17% and three 5%, so 18,327 impressions have more than one segment and 8,977 accumulate more watch time than the clip's own duration.

Follower counts follow a power law. Followees are chosen with Pareto weights, giving a maximum of 1,310 followers against a mean of 11, with the top 50 accounts holding 21% of all follows. Clips per creator use the same mechanism, with the busiest creator posting 670.

Activity follows a daily rhythm. Timestamps draw their hour from a 24-way weight vector that dips overnight and peaks between 18:00 and 21:00. The busiest hour carries 33,137 impressions against 1,784 in the quietest.

The agent layer is sparse where the brief says it is sparse. Sessions run 1 to 6 turns, mean 2.5. 20% of turns get a judge score and 3% get a user rating, so most turns have neither. 10% of judged turns are scored a second time by a different judge version, and 12% of rated turns are rated again with the rating flipped.

Moderation. Every clip gets a `pending` decision at upload and a second decision moments later, which is `live` 93% of the time. 6% receive a third decision days or weeks afterwards, giving 825 clips with a state change beyond the initial pair.

Prices and templates change over time. Each model has its rates re-priced every 90 to 150 days, and each prompt template is edited every 3 to 12 days, so a turn's cost and template text both depend on when the turn happened.

E.4 — JSON, used once

The one JSON column is `ToolCall.args_json`, guarded by `CHECK (json_valid(args_json))`.

Why this one. The three tools take different arguments: `search_videos(query, filters)`, `get_user_history(days)`, `fetch_trending_audio(region)`. Modelling them relationally means either a column per argument across all tools, leaving every row NULL in all but one or two, or a separate arguments table per tool. Both grow every time a tool is added, and the brief says tools are added. The shape differs per row, which is the case JSON is for.

Why nowhere else. Everything else in the schema has a fixed, known set of fields, which is not a case for JSON. Tool results were the other candidate and were rejected: `result` is stored as plain text because nothing in the brief queries inside it.

What is lost. No foreign key can point into the JSON, so `search_videos(video_id: 8821)` cannot reference Video. No CHECK can constrain its interior, so nothing stops `{"days": -5}` or a misspelled key. Reading it means `json_extract(args_json, '$.days')`, which cannot use an ordinary index. The `json_valid()` CHECK guarantees the text parses.

Deliverable F — Queries

Full SQL is in `queries.sql`. Runtimes are untuned and are the Assignment 2 baseline. Measured on the submission machine: macOS, SQLite 3.53.4.

Summary

Query	Rows	Runtime
F1 Top 10 audio tracks by distinct videos in the last 7 days	10	12 ms
F2 Watch hours and completion rate per creator, live clips only	1,250	966 ms
F3 Videos with no audio track, with NOT IN	0	4 ms
F3 The same question, with NOT EXISTS	3,018	5 ms
F4 Users who liked then retracted within 60 seconds	96	2 ms
F5 Videos whose caption carries a given hashtag	1,935	24 ms
F6 Users shown a creator's clips who never engaged	3,835	82 ms
F6 A signal roll-up with UNION	3	2 ms
F6 The same roll-up with UNION ALL	6,612	3 ms
F7 Cost of each agent session last month, by prompt template version	240	4 ms
F8 Videos whose moderation state changed more than twice	825	18 ms
F9 Each user's longest streak of consecutive active days	5,000	643 ms
F10 Creators by 7-day rolling watch time, with week-over-week change	194	281 ms
F11 Full nesting tree for one session's tool calls, with depth	12	4 ms
F12 Sessions where a recommended clip was watched to completion	54	17 ms
F13 Turns the judge scored above 4 but the user rated down	6	0 ms

F1 • Top 10 audio tracks by distinct videos in the last 7 days

```

SELECT t.track_id,
       t.name,
       t.source_kind,
       count(DISTINCT v.video_id) AS videos
FROM AudioTrack t
JOIN Video v ON v.audio_track_id = t.track_id
WHERE v.uploaded_at >= strftime('%Y-%m-%dT%H:%M:%SZ',

```

```

                                (SELECT max(uploaded_at) FROM Video), '-7 days')
GROUP BY t.track_id, t.name, t.source_kind
ORDER BY videos DESC, t.track_id
LIMIT 10;

```

Rows returned: 10 Runtime: 12 ms

track_id	name	source_kind	videos
78	bold biscuit sound	original	12
107	swift moth sound	original	12
224	velvet koi sound	original	11
26	loud gecko sound	original	10
58	plush waffle sound	original	10

Reading: the busiest sound carries 12 of the 1,872 clips uploaded that week, so no single track dominates at this scale.

F2 · Watch hours and completion rate per creator, live clips only

```

WITH current_state AS (
    SELECT d.video_id, d.state
    FROM ModerationDecision d
    JOIN (SELECT video_id, max(decided_at) AS latest
          FROM ModerationDecision
          GROUP BY video_id) m
    ON m.video_id = d.video_id AND m.latest = d.decided_at
),
live_video AS (
    SELECT v.video_id, v.owner_id, v.duration_ms
    FROM Video v
    JOIN current_state c ON c.video_id = v.video_id
    WHERE c.state = 'live'
),
per_impression AS (
    SELECT i.impression_id,
           lv.owner_id,
           sum(s.end_offset_ms - s.start_offset_ms) AS watch_ms,
           max(CASE WHEN s.end_offset_ms >= lv.duration_ms * 0.95 THEN 1 ELSE 0 END) AS completed
    FROM Impression i
    JOIN live_video lv ON lv.video_id = i.video_id
    LEFT JOIN ViewSegment s ON s.impression_id = i.impression_id
    GROUP BY i.impression_id, lv.owner_id
)
SELECT c.user_id AS creator_id,
       count(p.impression_id) AS impressions,
       ROUND(COALESCE(sum(p.watch_ms), 0) / 3600000.0, 3) AS watch_hours,
       ROUND(COALESCE(avg(p.completed), 0), 4) AS mean_completion_rate
FROM Creator c
LEFT JOIN per_impression p ON p.owner_id = c.user_id
GROUP BY c.user_id
ORDER BY watch_hours DESC, creator_id;

```

Rows returned: 1,250 Runtime: 966 ms

creator_id	impressions	watch_hours	mean_completion_rate
23	9005	26.237	0.021
2283	18851	22.583	0.0186

creator_id	impressions	watch_hours	mean_completion_rate
2020	15336	20.293	0.0218
4699	7139	19.722	0.0202
1683	8752	19.427	0.0194

Reading: the top creator holds 26 watch hours against a mean completion rate near 2%, which is the funnel working as the brief describes.

F3 · Videos with no audio track, with NOT IN

```
SELECT v.video_id, v.caption
FROM Video v
WHERE v.audio_track_id NOT IN (SELECT track_id FROM AudioTrack)
ORDER BY v.video_id;
```

Rows returned: 0 **Runtime:** 4 ms

No rows returned.

Reading: zero rows, which is wrong.

F3 · The same question, with NOT EXISTS

```
SELECT v.video_id, v.caption
FROM Video v
WHERE NOT EXISTS (SELECT 1 FROM AudioTrack t WHERE t.track_id = v.audio_track_id)
ORDER BY v.video_id;
```

Rows returned: 3,018 **Runtime:** 5 ms

video_id	caption
9	part 26 of the series #tutorial #viral
12	day 26 of trying
39	part 10 of the series #trending #asmr #prank
49	how it started vs how it ended #howto
58	look at this

Reading: 3,018 clips carry no track, which is the correct answer.

The difference. NOT IN evaluates audio_track_id NOT IN (list). For a clip with no track that is NULL NOT IN (...), which is NULL rather than true, so the row is not returned and the whole result is empty. NOT EXISTS asks whether a matching AudioTrack row exists; for a NULL key none does, so the row is returned. NOT IN is only safe when both the column and the subquery are free of NULLs.

F4 · Users who liked then retracted within 60 seconds

```
SELECT l.user_id,
       l.video_id,
       l.occurred_at AS liked_at,
       r.occurred_at AS retracted_at,
       CAST((julianday(r.occurred_at) - julianday(l.occurred_at)) * 86400 AS INT) AS seconds_held
FROM Signal l
JOIN Signal r
  ON r.user_id = l.user_id
 AND r.video_id = l.video_id
 AND r.signal_type = 'like_retracted'
```

```

AND r.occurred_at > l.occurred_at
WHERE l.signal_type = 'like'
AND (julianday(r.occurred_at) - julianday(l.occurred_at)) * 86400 <= 60
ORDER BY seconds_held, l.user_id;

```

Rows returned: 96 Runtime: 2 ms

user_id	video_id	liked_at	retracted_at	seconds_held
904	9899	2026-05-24T19:26:22Z	2026-05-24T19:26:25Z	2
4354	8411	2026-07-08T23:16:55Z	2026-07-08T23:16:58Z	2
1831	13792	2026-08-30T23:24:42Z	2026-08-30T23:24:45Z	3
2574	3546	2026-07-19T16:37:50Z	2026-07-19T16:37:53Z	3
4368	18721	2026-05-08T19:03:59Z	2026-05-08T19:04:03Z	3

Reading: 96 pairs, most within 3 to 20 seconds, which suggests accidental taps.

F5 · Videos whose caption carries a given hashtag

```

WITH target(tag) AS (VALUES ('viral'))
SELECT v.video_id, v.caption
FROM Video v, target
WHERE ' ' || replace(replace(replace(replace(lower(trim(v.caption)),
    ',', ' '), '.', ' '), '!', ' '), '?', ' '), '#', ' ') || ' '
    LIKE '% ' || lower(target.tag) || ' %'
ORDER BY v.video_id;

```

Rows returned: 1,935 Runtime: 24 ms

video_id	caption
9	part 26 of the series #tutorial #viral
16	how it started vs how it ended #outfit #viral #duet
19	this took forever #outfit #viral #cover
24	nobody talks about this #howto #viral #trending
28	how it started vs how it ended #outfit #viral #trending

Reading: 1,935 of 20,000 clips carry the tag, matching a plain LIKE check on the same data.

LIKE or GLOB. I used LIKE, which is case-insensitive for ASCII. GLOB is case-sensitive and would need `lower()` on both sides. For a caption in Tamil the answer is the same either way, since Tamil has no case distinction.

F6 · Users shown a creator's clips who never engaged

```

SELECT i.user_id
FROM Impression i
JOIN Video v ON v.video_id = i.video_id
WHERE v.owner_id = 2283
EXCEPT
SELECT s.user_id
FROM Signal s
JOIN Video v ON v.video_id = s.video_id
WHERE v.owner_id = 2283;

```

Rows returned: 3,835 Runtime: 82 ms

user_id
1
2
3
4
5

Reading: 3,835 of the users shown this creator's clips never signalled on any of them.

F6 · A signal roll-up with UNION

```
SELECT signal_type FROM Signal WHERE signal_type IN ('like', 'save')
UNION
SELECT signal_type FROM Signal WHERE signal_type IN ('save', 'share');
```

Rows returned: 3 **Runtime:** 2 ms

signal_type
like
save
share

Reading: three rows: like, save, share.

F6 · The same roll-up with UNION ALL

```
SELECT signal_type FROM Signal WHERE signal_type IN ('like', 'save')
UNION ALL
SELECT signal_type FROM Signal WHERE signal_type IN ('save', 'share');
```

Rows returned: 6,612 **Runtime:** 3 ms

signal_type
like
like
save
like
like

Reading: 6,612 rows.

The difference. UNION removes duplicates across both arms and returns the three distinct signal types. UNION ALL keeps every row, so 6,612 is the sum of both arms including the `save` rows counted twice, once from each. UNION also has to sort or hash to deduplicate, which UNION ALL does not.

F7 · Cost of each agent session last month, by prompt template version

```
WITH turn_cost AS (
  SELECT t.session_id,
         t.template_version_id,
         (t.input_tokens * p.input_rate
          + t.output_tokens * p.output_rate
          + t.cached_tokens * p.cached_rate) / 1000.0 AS cost
  FROM Turn t
```

```

JOIN ModelPrice p
  ON p.model_id = t.model_id
  AND t.occurred_at >= p.valid_from
  AND (p.valid_to IS NULL OR t.occurred_at < p.valid_to)
WHERE t.assistant_message IS NOT NULL
  AND strftime('%Y-%m', t.occurred_at)
    = strftime('%Y-%m', (SELECT max(occurred_at) FROM Turn))
)
SELECT session_id,
       template_version_id,
       count(*) AS turns,
       ROUND(sum(cost), 4) AS cost
FROM turn_cost
GROUP BY session_id, template_version_id
HAVING sum(cost) > 0.05
ORDER BY cost DESC, session_id;

```

Rows returned: 240 Runtime: 4 ms

session_id	template_version_id	turns	cost
909	36	6	2.9931
243	36	5	2.8352
1840	36	5	2.6584
350	36	5	2.3779
1428	36	6	2.349

Reading: 240 session-versions cost more than 0.05, nearly all on the newest template version.

F8 · Videos whose moderation state changed more than twice

```

SELECT video_id,
       count(*) AS decisions,
       group_concat(state, ' -> ' ORDER BY decided_at) AS transitions
FROM ModerationDecision
GROUP BY video_id
HAVING count(*) > 2
ORDER BY decisions DESC, video_id;

```

Rows returned: 825 Runtime: 18 ms

video_id	decisions	transitions
25	3	pending -> live -> demoted
62	3	pending -> live -> live
67	3	pending -> live -> live
79	3	pending -> live -> demoted
90	3	pending -> live -> taken_down

Reading: 825 clips have a third decision after the initial pending and live pair.

`group_concat` accepts `ORDER BY` only from SQLite 3.44. On anything older the output order is undefined and the sequence would look plausible while being wrong.

F9 · Each user's longest streak of consecutive active days

```

WITH active AS (
  SELECT DISTINCT user_id, date(shown_at) AS day

```

```

    FROM Impression
),
grouped AS (
    SELECT user_id, day,
           julianday(day) - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day) AS grp
    FROM active
),
runs AS (
    SELECT user_id, grp, count(*) AS streak, min(day) AS from_day, max(day) AS to_day
    FROM grouped
    GROUP BY user_id, grp
)
SELECT user_id, max(streak) AS longest_streak
FROM runs
GROUP BY user_id
ORDER BY longest_streak DESC, user_id;

```

Rows returned: 5,000 Runtime: 643 ms

user_id	longest_streak
597	243
1082	243
1099	243
1270	243
2293	243

Reading: nine users were active on all 243 days in the data, and these are the heaviest viewers at up to 34 impressions a day.

F10 · Creators by 7-day rolling watch time, with week-over-week change

```

WITH daily AS (
    SELECT v.owner_id AS creator_id,
           date(i.shown_at) AS day,
           sum(s.end_offset_ms - s.start_offset_ms) / 3600000.0 AS hours
    FROM ViewSegment s
    JOIN Impression i ON i.impression_id = s.impression_id
    JOIN Video v ON v.video_id = i.video_id
    GROUP BY v.owner_id, date(i.shown_at)
),
rolling AS (
    SELECT creator_id, day, hours,
           sum(hours) OVER (PARTITION BY creator_id
                           ORDER BY julianday(day)
                           RANGE BETWEEN 6 PRECEDING AND CURRENT ROW) AS hours_7d
    FROM daily
),
wow AS (
    SELECT creator_id, day, hours_7d,
           LAG(hours_7d, 7) OVER (PARTITION BY creator_id ORDER BY julianday(day)) AS prev_7d
    FROM rolling
)
SELECT creator_id,
       day,
       ROUND(hours_7d, 3) AS hours_7d,
       ROUND(hours_7d - prev_7d, 3) AS wow_change,
       RANK() OVER (PARTITION BY day ORDER BY hours_7d DESC) AS rank_on_day

```

```

FROM wow
WHERE day = (SELECT max(date(shown_at)) FROM Impression)
ORDER BY rank_on_day;

```

Rows returned: 194 Runtime: 281 ms

creator_id	day	hours_7d	wow_change	rank_on_day
23	2026-08-31	0.743	-0.126	1
2020	2026-08-31	0.639	0.083	2
827	2026-08-31	0.617	0.15	3
2283	2026-08-31	0.596	-0.068	4
1683	2026-08-31	0.575	0.076	5

Reading: the leader accumulated 0.74 watch hours over the week and is down 0.13 on the week before.

Seven calendar days, not seven active days. A creator who posted nothing for three days should show the dip, which is what Growth asked for. The RANGE frame over julianday(day) gives calendar days; ROWS BETWEEN 6 PRECEDING would have counted only days that have rows. SQLite's RANGE frames take numeric offsets only, so there is no INTERVAL '7 days' and ordering by julianday() is required.

F11 · Full nesting tree for one session's tool calls, with depth

```

WITH RECURSIVE tree AS (
  SELECT c.call_id, c.turn_id, c.tool_name, c.latency_ms, c.errorred,
         0 AS depth,
         printf('%08d', c.call_id) AS path
  FROM ToolCall c
  JOIN Turn t ON t.turn_id = c.turn_id
  WHERE t.session_id = 164 AND c.parent_call_id IS NULL
  UNION ALL
  SELECT c.call_id, c.turn_id, c.tool_name, c.latency_ms, c.errorred,
         tree.depth + 1,
         tree.path || '.' || printf('%08d', c.call_id)
  FROM ToolCall c
  JOIN tree ON c.parent_call_id = tree.call_id
)
SELECT turn_id,
       depth,
       substr(' ', 1, depth * 2) || tool_name AS call_tree,
       latency_ms,
       errorred
FROM tree
ORDER BY path;

```

Rows returned: 12 Runtime: 4 ms

turn_id	depth	call_tree	latency_ms	errorred
396	0	search_videos	128	0
396	1	fetch_trending_audio	334	0
397	0	fetch_trending_audio	317	0
397	1	search_videos	367	0
397	0	fetch_trending_audio	175	0

Reading: the deepest chain is search_videos calling fetch_trending_audio, and most calls sit at the top level.

F12 · Sessions where a recommended clip was watched to completion

```
WITH completed AS (  
    SELECT i.impression_id,  
           i.recommendation_id,  
           sum(s.end_offset_ms - s.start_offset_ms) AS watch_ms  
    FROM Impression i  
    JOIN ViewSegment s ON s.impression_id = i.impression_id  
    JOIN Video v ON v.video_id = i.video_id  
    WHERE i.recommendation_id IS NOT NULL  
    GROUP BY i.impression_id, i.recommendation_id, v.duration_ms  
    HAVING max(s.end_offset_ms) >= max(v.duration_ms) * 0.95  
)  
SELECT t.session_id,  
       t.seq AS turn_seq,  
       r.position AS shelf_position,  
       r.video_id,  
       c.watch_ms,  
       i.shown_at  
FROM completed c  
JOIN Recommendation r ON r.recommendation_id = c.recommendation_id  
JOIN Turn t ON t.turn_id = r.turn_id  
JOIN Impression i ON i.impression_id = c.impression_id  
ORDER BY t.session_id, t.seq, r.position;
```

Rows returned: 54 Runtime: 17 ms

session_id	turn_seq	shelf_position	video_id	watch_ms	shown_at
17	4	2	9776	20139	2026-02-27T18:57:20Z
35	1	4	14041	130950	2026-08-13T18:53:03Z
56	3	3	7276	83578	2026-04-21T20:48:00Z
89	2	4	12514	67453	2026-08-31T16:59:27Z
103	5	4	12779	43497	2026-08-07T12:43:52Z

Reading: 54 recommendations were watched to the end, and positions 2, 3 and 4 appear as often as position 1.

F13 · Turns the judge scored above 4 but the user rated down

```
SELECT j.turn_id,  
       j.judge_version_id,  
       j.helpfulness,  
       j.groundedness,  
       j.safety,  
       u.rating,  
       u.rated_at  
FROM JudgeScore j  
JOIN UserRating u ON u.turn_id = j.turn_id  
WHERE j.helpfulness > 4  
      AND u.rating = 'down'  
      AND u.rated_at = (SELECT max(rated_at) FROM UserRating u2 WHERE u2.turn_id = j.turn_id)  
ORDER BY j.helpfulness DESC, j.turn_id;
```

Rows returned: 6 Runtime: 0 ms

turn_id	judge_version_id	helpfulness	groundedness	safety	rating	rated_at
1765	1	4.64	4.18	4.81	down	2026-06-17T19:29:10Z
2725	1	4.47	3.77	4.59	down	2026-05-06T17:49:02Z
3529	3	4.29	3.1	4.83	down	2026-04-11T20:07:37Z
4278	1	4.28	4.26	4.6	down	2026-06-02T21:44:50Z
1784	1	4.12	2.37	4.33	down	2026-02-28T00:16:18Z

Reading: six turns where the judge was confident and the user disagreed.

Why this set is the most commercially valuable data in the company. It is the only direct evidence of where the automated quality metric is wrong. Every dashboard, regression test and prompt iteration is scored by the judge, so a judge that is confidently wrong makes the whole quality process lie in the same direction without anyone noticing. These rows are labelled counterexamples that can be used to retrain the judge, and they cost nothing to collect because the user volunteered the label.

Deliverable G — Views and Transactions

G.1 — View specification

View	Consumer	Exposes	Hides	Hides which B.2 decision	Updatable
v_public_profile	mobile client	user_id, handle, display_name, follower_count	phone, email, account_state, deletion_requested_at	none	expected yes
v_video_current_state	Trust and Safety	video_id, state, decided_at, reviewer_id	the decision history behind it	moderation state is an append-only log	expected no
v_creator_tier_current	Growth	creator_id, tier, valid_from	earlier tier periods	creator tier uses validity intervals	expected yes
v_video_daily_engagement	Growth analysts	video_id, day, impressions, views, watch_seconds, net_likes	the impression, segment and signal tables	likes are an append-only stream	expected no
v_turn_cost	Finance	turn_id, session_id, occurred_at, model_id, token counts, cost	the price table and the as-of join	pricing uses validity intervals	expected no

Row counts on the generated data:

View	Rows	Check
v_public_profile	4,566	active accounts only; 0 rows for any non-active account
v_video_current_state	20,000	exactly one row per video
v_creator_tier_current	1,250	exactly one row per creator
v_video_daily_engagement	206,651	140,793 of these are days with impressions but no views and no likes, reported as zeros
v_turn_cost	4,701	one row per turn that produced an assistant message

Testing the claim that a view hides its storage. Moderation state is an append-only log in B.2. Switching it to validity intervals, or to a `current_state` column on Video, would change the body of `v_video_current_state` and nothing else: Trust and Safety query `SELECT state FROM v_video_current_state WHERE video_id = ?` in every case. The same holds for `v_creator_tier_current` and `v_turn_cost` if tier or pricing changed strategy.

`v_video_daily_engagement` is the view that saves its consumer the most work. Days with impressions but no engagement appear with zeros rather than being dropped, which takes three separate aggregates joined back to the impression days. An analyst writing that by hand would use inner joins and lose the 140,793 zero-engagement days without noticing.

G.2 (a) — Updatability

Attempting to write through the view:

```
INSERT INTO v_public_profile (handle, display_name, follower_count)
VALUES ('newhandle', 'New User', 0);
```

Error: cannot modify v_public_profile because it is a view

The same error comes back for UPDATE and DELETE. SQLite views are read-only unless an INSTEAD OF trigger is defined.

A view is updatable in principle when the mapping from base rows to view rows is one-to-one and reversible, so a write through the view resolves to a write against exactly one identifiable base row. `v_public_profile` meets that: it reads one table with no aggregate, no GROUP BY and no set operation, so each of its rows is one AppUser row and an INSERT maps to one AppUser insert. PostgreSQL would accept it on that basis. `v_video_daily_engagement` fails it, because one of its rows is an aggregate over many impressions, segments and signals grouped by video and day. Writing `impressions = 40` into such a row has no defined meaning, since more than one set of base rows produces that total and the engine cannot know which to change. Aggregation is many-to-one and a many-to-one mapping has no unique inverse, so no engine can derive the write automatically. Whoever writes the view can still supply that inverse by hand in an INSTEAD OF trigger, which both engines support, but that is a person defining the mapping rather than the engine deriving it.

G.2 (b) — The as-of problem

`v_creator_tier_current` works because “now” is not a parameter: its body says `WHERE valid_to IS NULL`, a fixed predicate that holds for every creator. Tier as of a past timestamp `T` needs `WHERE valid_from <= T AND (valid_to IS NULL OR valid_to > T)`, and `T` has to come from the caller. A view has no parameters; it is a stored query expression the engine expands in place, so there is nowhere for `T` to enter. Hard-coding a date works once and is wrong the next day, and exposing `valid_from` and `valid_to` for the ML team to filter defeats the point of the view, since the consumer is then back to knowing that tier is stored as intervals.

What I would use instead is a table-valued function, which is a view that takes arguments. PostgreSQL supports this directly:

```
CREATE FUNCTION creator_tier_asof(t timestampz)
RETURNS TABLE (creator_id bigint, tier text) AS $$
    SELECT user_id, tier FROM CreatorTierPeriod
    WHERE valid_from <= t AND (valid_to IS NULL OR valid_to > t)
$$ LANGUAGE sql STABLE;
```

called as `SELECT * FROM creator_tier_asof('2026-03-03T14:32:07Z')`. The consumer passes a timestamp and still never learns how the tier is stored.

SQLite has no equivalent in plain SQL. The options are a virtual table, a table-valued function registered through the C or Python API, or letting this one consumer write the predicate itself against a documented interval table. Since the ML team's extraction runs in Python, registering the function on the connection is the best of the three.

G.3 — Transactions

Full scripts are in `transactions.sql`; `run_transactions.py` drives the parts that need more than one connection.

A note on injecting failures. The task sheet's template uses `SELECT 1/0` as the deliberate failure. SQLite returns NULL for division by zero rather than raising, so that statement succeeds, the transaction commits, and the demonstration proves the opposite of what it intends. I checked: `SELECT 1/0`, `SELECT 1.0/0` and `SELECT 1%0` all return NULL. Each script below injects a statement that violates a real constraint instead.

T1 — a block must break follows in both directions

The task sheet describes T1 as two writes: end the like row, then write a negative-signal row, and show that a failure between them leaves neither. That does not apply here, because of the assumption made in A4. Likes and retractions are a single append-only stream, so a retraction is one insert into Signal and the like row is never updated. With only one write there is no second write to fail after, so the failure the task sheet asks us to demonstrate cannot occur in this schema.

T1 therefore runs on the operation in this schema that is two writes for one action. Since we assumed in A12 that a block ends any follows in both directions, blocking someone inserts a Block row and updates the Follow rows, which is the same structure as the retraction the task sheet describes and is open to the same failure. The retraction is shown below it as a single insert, for contrast.

```
before: 2 open follows, 0 blocks
injected failure: CHECK constraint failed: blocker_id <> blocked_id
after: 2 open follows, 0 blocks
consistent: True
```

The failure lands after the block was inserted and after the follows were ended. Both are rolled back, so neither the block nor the ended follows survive. Without the transaction the database would hold a block whose follows are still open, which is the shape of the founders' weekly failure.

And the retraction, for contrast: one insert, with the current like state read back as `like_retracted` from the latest signal. No transaction needed.

T2 — an uncommitted moderation decision is invisible

```
journal mode: wal
reader before:          live
writer inside its own tx: taken_down
reader while uncommitted: live
third connection write: database is locked
reader after commit:    taken_down
```

The writer sees its own uncommitted insert. A second connection keeps seeing `live` until the commit lands, then sees `taken_down`. This is what `journal_mode = WAL` buys: in the default rollback-journal mode the reader would block and wait instead, which demonstrates locking rather than isolation.

A third connection attempting its own write while the transaction is open gets `database is locked`, which is `SQLITE_BUSY`.

What `SQLITE_BUSY` tells us. SQLite permits one writer at a time across the whole database, so write transactions are serialised rather than interleaved. A multi-writer engine must let two transactions write concurrently and then detect conflicts, which is what row-level locking and multiversion concurrency control are for.

The anomaly sidestepped is the **lost update**: two transactions read the same row, each computes a new value from what it read, and each writes it back, so the second overwrite discards the first one's change. Under SQLite's single-writer rule the second transaction cannot start writing until the first commits, so it reads the updated value and the lost update cannot occur.

T3 — a handle changes at most twice in any rolling 365 days

A13 chose a rolling window, which counts history and so cannot be a CHECK, because a CHECK sees only the row being inserted. It is enforced by a BEFORE INSERT trigger that counts prior changes inside the window and calls RAISE (ABORT, ...).

```
change to alicia: accepted
change to alison: accepted
change to alyssa: rejected, handle changed twice already in the last 365 days
handle now: alison
history:      ['alice', 'alicia', 'alison']
consistent: True
```

Each change is two writes: a HandleChange row and an update to AppUser.handle. The third attempt aborts on the first of them, and the transaction rolls back, so the handle on AppUser still matches the last row in the history. Without the transaction the abort could leave the AppUser row updated with no matching history row, which would then miscount the next attempt.

What the trigger cannot guarantee. It reads the count at insert time, so two concurrent inserts could each see two prior changes and both be allowed, producing three. SQLite's single-writer rule closes that gap here. On a multi-writer engine the read and the insert would have to be serialised explicitly.

Appendix — Use of an LLM assistant

Section 0.4 permits LLM assistants and asks for a note on where one was used and at least one instance where its suggestion was wrong.

Where I used one

Deliverable	What I used it for
A	Pressure-testing candidate assumptions against the reversal question, and discarding the ones whose opposite changed nothing
B	Drafting the Mermaid for the five diagrams, then correcting cardinalities by hand against the relationship inventory
C	Drafting the relation listings from the ER model, and checking the relational algebra expressions matched the SQL
D	Checking closure computations and candidate-key derivations
E	Drafting DDL from the C.1 listings, and writing the generator's distribution code
F	Drafting queries, then running each one and checking the row count against what I expected
G	Drafting the views, and building the two-connection transaction harness

Where it was wrong

1. INSERT OR IGNORE hid a real constraint failure

The generator produced occasional duplicate keys from random collisions, and the assistant suggested INSERT OR IGNORE to absorb them. It worked, and the load completed with no errors.

INSERT OR IGNORE suppresses CHECK violations as well as uniqueness ones. View segment durations are computed in milliseconds while timestamps are stored to second precision, so any segment shorter than a second produced ended_at = started_at and failed CHECK (ended_at > started_at). Roughly 400 rows a run were being dropped without any error.

How I caught it. I asked the assistant to remove OR IGNORE and generate unique keys properly instead. The load then failed immediately on CHECK constraint failed: ended_at > started_at. The fix was to round the wall-clock duration up to a minimum of one second while leaving the playhead offsets in milliseconds. Recorded as R7 and R8 in the A.3 log.

2. An invented creator tier ladder

The C.1 listing for `CreatorTierPeriod` came back with `tier : varchar(16) -- 'none' | 'bronze' | 'silver' | 'gold' | 'partner'`, presented in the same format as the value sets taken from the brief.

The brief names no tiers anywhere. Section 2.2 says only that a creator sits in a monetisation tier that changes over time.

How I caught it. I asked where the five values came from. They were invented. Reading them in context they look like they were quoted from the brief, which would have been a fabricated requirement sitting in the logical design. Replaced with a three-value set and flagged as a choice rather than a given.

3. A miscounted composite key, twice

The C.2 log claimed JudgeScore's natural composite key was four columns deep, and separately that Recommendation's would put three columns into Impression. Both are one column too many: JudgeScore is (`session_id`, `seq`, `judge_version_id`), which is three, and Recommendation's is (`turn_id`, `position`) once Turn carries a surrogate, which puts two into Impression.

How I caught it. I asked the assistant to explain the claim, and counting the columns while it did showed the arithmetic was wrong. Both traced to the same cause: the figure was carried over from an earlier draft where Turn still used its natural composite key, and was not recomputed after the surrogate was introduced.

4. An unnecessary lookup in the D.2 trigger

The D.2 write-up argued that a `like_count` trigger would need to read the user's previous signal on that clip before it could know whether to increment, and that this reintroduced the cost the column was meant to avoid.

That is wrong. The signal type alone determines the direction: `like` adds one, `like_retracted` subtracts one. No lookup is needed.

How I caught it. The reasoning did not follow when I read it back, and the assistant agreed on being asked. The paragraph was rewritten, which also made D.2's conclusion weaker and more honest, since the enforcement turns out to be less baroque than first claimed.

5. A find-and-replace that broke the SQL

While reformatting `queries.sql`, the assistant ran a global replace of " * " to change a separator character in the comment headings. The same three characters appear in `duration_ms * 0.95` inside F2 and F12, so the replace corrupted the multiplication into a syntax error.

How I caught it. Re-running every statement in `queries.sql` against a freshly rebuilt database, which is a step worth repeating after any bulk edit. The replace was redone restricted to comment lines only.

Two things the assistant was right about that the task sheet was not

`SELECT 1/0` is the failure injection suggested in the G.3 template. SQLite returns NULL for division by zero rather than raising, so the statement succeeds, the transaction commits, and the demonstration proves the opposite of what it intends. Verified directly: `SELECT 1/0`, `SELECT 1.0/0` and `SELECT 1%0` all return NULL. Each script injects a real constraint violation instead.

`datetime(x, '-7 days')` returns '2026-08-24 22:46:09' with a space where the stored ISO-8601 format has a T. Compared as text, 'T' sorts above a space, so every row from earlier on the boundary day slips into the window. In F1 the obvious version returned 2,115 videos against a correct 1,872, a silent 13% overcount. All date arithmetic uses `strftime('%Y-%m-%dT%H:%M:%SZ', ...)` instead.